

Real Estate Price Prediction

Ames Housing Dataset — Comprehensive Analysis

The Extractors | Project Apex | Nov 2025

This report documents the full end-to-end data science pipeline applied to the **Ames Housing Dataset** for residential property price prediction. It covers data cleaning & imputation, outlier handling, feature engineering, encoding, VIF-based multicollinearity analysis, feature scaling, and comparative model evaluation across Simple Linear, Multiple Linear, Ridge, Lasso, and Random Forest Regression.

Table of Contents

1. [House] ****Problem Statement: Real Estate Price Prediction****
2. [Plant] ****Task: Now we are going handle Outliers in All Continuous Variables Accordingly****
3. [Search] ****Checking for the Following Points and Fixing Them :****
4. [Down]
5. [Down]
6. Review column names
7. [Down]
8. [Green] ****Now We Are Going for Feature Scaling****
9. [Target] ****Final Correlation Validation using Scatter Plots****
10. [No] ****Time Series Plots Are Not Applicable****
11. [Search] ****Exploratory Data Analysis (EDA) Summary****
12. **** Sequential Summary of Work Done on Dataset****
13. Now Instantiating and Train Simple Linear Regression Model
14. ****Evaluating Simple Linear Regression Model****
15. Summary: Simple Linear Regression Model Performance
16. [Art]Visualize Simple Linear Regression Predictions
17. [Gear] ****Instantiate Linear Regression Model****
18. [Chart] ****Train-Test Split Data for Linear Regression Model****
19. ****The Linear Regression model has been trained successfully****
20. ****Final Model Evaluation Summary****

[House] Problem Statement: Real Estate Price Prediction

The objective of this project is to **predict the sale price of residential properties** based on various structural, qualitative, and locational features.

By analyzing the **Ames Housing Dataset**, we aim to identify key factors that influence house prices, handle data irregularities through preprocessing, and build a **robust predictive model** capable of accurately estimating real estate values.

[Target] Objective

To perform **comprehensive data preprocessing, exploratory data analysis (EDA), feature transformation, and model building**

that together help in understanding the **driving factors of house prices** and achieving **high predictive accuracy**.

[Books] Importing the Necessary Libraries and Checking the DataFrame [Down]

```
[1] Python
```

```
import pandas as pd
import numpy as np
```

```
[2] Python
```

```
# Load the dataset into a pandas DataFrame
df = pd.read_csv('LatestAmesHousing.csv')

# Display the first 5 rows of the DataFrame
display(df.head())
```

[Clipboard] Displaying the Data Types of All Columns [Down]

```
[3] Python
```

```
# Display the data types of each column
display(df.info())
```

[Chart] Checking the Descriptive Statistics of the DataFrame [Down]

```
[4] Python
```

```
# Show descriptive statistics for the DataFrame
display(df.describe())
```

[Search] Now Checking Missing Values in the DataFrame — Findings Will Be Handled Accordingly [Down]

```
[5] Python
```

```
# Check total missing values in each column and sort in descending order
missing_values = df.isnull().sum()
display(missing_values[missing_values > 0].sort_values(ascending=False))
```

[Chart] Visualization of Missing Value Percentage Across All Columns — Identifying High-Missing Features [Down]

[6] Python

```
import matplotlib.pyplot as plt
import seaborn as sns

# Calculate the percentage of missing values
missing_percentage = (df.isnull().sum() / len(df)) * 100

# Filter out columns with no missing values
missing_percentage = missing_percentage[missing_percentage > 0].sort_values(ascending=False)

# Create a bar plot
plt.figure(figsize=(12, 6))
sns.barplot(x=missing_percentage.index, y=missing_percentage, color='lightsalmon')
# 'lightsalmon' is a light orange color
plt.xticks(rotation=90)
plt.xlabel("Columns")
plt.ylabel("Percentage of Missing Values")
plt.title("Percentage of Missing Values per Column")
plt.tight_layout()
plt.show()
```

[7] Python

```
# Show columns with more than 60% missing values
display(missing_percentage[missing_percentage > 60])
```

[Cut] Dropping Columns Having More Than 60% Missing Values [Down]

[8] Python

```
# List of columns to drop
columns_to_drop = ['Pool QC', 'Misc Feature', 'Alley', 'Fence', 'Mas Vnr Type']

# Drop the columns from the DataFrame
df_dropped = df.drop(columns=columns_to_drop)

# Display the first 5 rows of the updated DataFrame
display(df_dropped.head())
```

[Search] Analyzing Columns Having Remaining Missing Values Below 60% [Down]

[9] Python

```
# Select only numerical columns
numerical_cols = df_dropped.select_dtypes(include=np.number)

# Check for missing values in numerical columns
missing_numerical_values = numerical_cols.isnull().sum()

# Filter for numerical columns with missing values and display in descending order
missing_numerical_values = missing_numerical_values[missing_numerical_values > 0].sort_values(ascending=False)

display(missing_numerical_values)
```

[Search] Now Visualizing Missing Values in Numerical Columns Using Histogram [Down]

[10] Python

```
import matplotlib.pyplot as plt
import seaborn as sns

columns_to_plot = ['Lot Frontage', 'Garage Yr Blt', 'Mas Vnr Area', 'Bsmt Half Bath', 'Bsmt Full Bath', 'BsmtFin SF 1', 'BsmtFin SF 2', 'Total Bsmt SF', 'Bsmt Unf SF', 'Garage Cars', 'Garage Area']

for col in columns_to_plot:
    plt.figure(figsize=(8, 5))
    sns.histplot(df_dropped[col].dropna(), kde=True)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.show()
```

[11] Python

```
# Check missing values in GarageType and GarageYrBlt
df[['Garage Type', 'Garage Yr Blt']].isnull().sum()
```

[Note] Note: Logical Handling of Related Missing Values

We proceeded **logically** —

if the **'Garage Type'** column has missing values similar to **'Garage Yr Blt'**, it indicates that those houses **do not have a garage**.

Hence, the missing values in such columns are **filled with 0** to represent the **absence of a garage** accurately. [OK]

[Puzzle] Filling Missing Values in Numerical Columns

The following numerical columns contain missing values, which are filled as per their logical and statistical relevance:

- **Lot Frontage:** Filled with **Median**,

because missing values here **do not imply absence of lot frontage**, hence median is the best representative value.

- **Garage Yr Blt:** 0
- **Mas Vnr Area:** 0
- **Bsmt Half Bath:** 0
- **Bsmt Full Bath:** 0
- **BsmtFin SF 1:** 0
- **BsmtFin SF 2:** 0
- **Total Bsmt SF:** 0
- **Bsmt Unf SF:** 0
- **Garage Cars:** 0
- **Garage Area:** 0

[Brain] **Reasoning:**

For columns **2–11**, a large number of values are already 0, indicating that many houses **do not have these amenities** (like garage or basement).

Hence, filling missing values with 0 is the **most appropriate and consistent approach**. [OK]

[12] Python

```
# Impute missing values in 'Lot Frontage' with the median
df_dropped['Lot Frontage'] = df_dropped['Lot Frontage'].fillna(df_dropped['Lot Frontage'].median())

# Impute missing values in specified columns with 0
zero_impute_cols = ['Garage Yr Blt', 'Mas Vnr Area', 'Bsmt Half Bath', 'Bsmt Full Bath',
                    'BsmtFin SF 1', 'BsmtFin SF 2', 'Total Bsmt SF', 'Bsmt Unf SF',
                    'Garage Cars', 'Garage Area']
for col in zero_impute_cols:
    df_dropped[col] = df_dropped[col].fillna(0)

# Verify that there are no more missing values in these columns
display(df_dropped[zero_impute_cols + ['Lot Frontage']].isnull().sum())
```

[13] Python

```
# Select only numerical columns from the updated DataFrame
numerical_cols_updated = df_dropped.select_dtypes(include=np.number)

# Check for missing values in the updated numerical columns
missing_numerical_values_updated = numerical_cols_updated.isnull().sum()

# Display the columns with missing values (if any)
display(missing_numerical_values_updated[missing_numerical_values_updated > 0])
```

[Check] Missing Values Have Been Successfully Filled with Appropriate Values in Numerical Columns

[Search] Now Checking for Missing Values in Categorical Columns [Down]

[14] Python

```
# Select only categorical columns from the updated DataFrame
categorical_cols = df_dropped.select_dtypes(include='object')

# Check for missing values in categorical columns
missing_categorical_values = categorical_cols.isnull().sum()

# Filter for categorical columns with missing values and display in descending order
missing_categorical_values = missing_categorical_values[missing_categorical_values > 0].sort_values(ascending=False)
display(missing_categorical_values)
```

[Search] Now visualizing Missing Values in Categorical Columns by plotting boxplots [Down]

[15] Python

```
import matplotlib.pyplot as plt
import seaborn as sns

# Get the list of categorical columns with missing values
categorical_cols_with_missing = missing_categorical_values.index.tolist()

# Create a countplot for each categorical column with missing values
for col in categorical_cols_with_missing:
    plt.figure(figsize=(10, 6))
    sns.countplot(data=df_dropped, y=col, order=df_dropped[col].value_counts().index,
                  palette='viridis')
```

```
plt.title(f'Countplot of {col}')
plt.xlabel('Count')
plt.ylabel(col)
plt.tight_layout()
plt.show()
```

[Note] Note: Handling Structural Missing Values in Categorical Columns

The following categorical columns:

Fireplace Qu, Garage Cond, Garage Qual, Garage Finish, Garage Type,
Bsmt Exposure, BsmtFin Type 2, Bsmt Cond, Bsmt Qual, BsmtFin Type 1

are being **filled with the value 'None'**,

as their missing values **indicate the absence of that specific amenity** (e.g., no fireplace, no garage, or no basement). [Down]

[16] Python

```
# List of categorical columns to impute with 'None'
categorical_cols_to_impute_none = ['Fireplace Qu', 'Garage Cond', 'Garage Qual',
'Garage Finish',
'Garage Type', 'Bsmt Exposure', 'BsmtFin Type 2', 'Bsmt Cond',
'Bsmt Qual', 'BsmtFin Type 1']

# Impute missing values in these columns with 'None'
for col in categorical_cols_to_impute_none:
df_dropped[col] = df_dropped[col].fillna('None')

# Verify that there are no more missing values in these columns
display(df_dropped[categorical_cols_to_impute_none].isnull().sum())
```

[Note] Note: Handling Missing Values in 'Electrical' Column

We will now handle the missing value in the **'Electrical'** column by filling it with the **mode value ('SBrkr' – Standard Circuit Breakers & Romex wiring)**,

because it is **not practically possible** for any house to exist **without an electricity connection**. [Down]

[17] Python

```
# Impute missing values in 'Electrical' with the mode
# Calculate the mode
electrical_mode = df_dropped['Electrical'].mode()[0]

# Fill missing values with the mode
df_dropped['Electrical'] = df_dropped['Electrical'].fillna(electrical_mode)

# Verify that there are no more missing values in 'Electrical'
display(df_dropped['Electrical'].isnull().sum())
```

[18] Python

```
# Check the total number of missing values in the entire DataFrame
total_missing_values = df_dropped.isnull().sum().sum()

# Display the total count of missing values
display(f"Total missing values in the DataFrame: {total_missing_values}")
```

[Puzzle] All Missing Values Have Been Successfully Handled Accordingly [Check]

[Search] Checking for Duplicate Values in the DataFrame [Down]

[19] Python

```
# Check for duplicate rows
duplicate_rows = df_dropped.duplicated()
display(f"Number of duplicate rows: {duplicate_rows.sum()}")

# Check for duplicate columns
duplicate_columns = df_dropped.columns.duplicated()
display(f"Are there any duplicate column names: {duplicate_columns.any()}")
```

[Check]No duplicate values are seen in the dataframe.

[20] Python

```
# Save the DataFrame to a CSV file
df_dropped.to_csv('cleaned_dataset.csv', index=False)

# Save the DataFrame to an Excel file
df_dropped.to_excel('cleaned_dataset.xlsx', index=False)

display("DataFrame saved to 'cleaned_dataset.csv' and 'cleaned_dataset.xlsx'")
```

[Green] Now We Are Going to Segregate Continuous and Discrete Variables

We will now **separate continuous and discrete variables** from the numerical columns, so that we can **analyze and handle outliers** in each category **independently and more effectively**. [Down]

[21] Python

```
# Get a list of all numerical columns in the DataFrame
numerical_cols = df_dropped.select_dtypes(include=np.number).columns.tolist()

# Display the list of numerical columns
display("All numerical columns:", numerical_cols)
```

[Chart] Continuous and Discrete Variables Overview

[Calc] Continuous Variables:

Lot Frontage, Lot Area, Mas Vnr Area, BsmtFin SF 1, BsmtFin SF 2, Bsmt Unf SF, Total Bsmt SF, 1st Flr SF, 2nd Flr SF, Gr Liv Area, Garage Area, Wood Deck SF, Open Porch SF, Enclosed Porch, Screen Porch, Pool Area.

[Num] Discrete Variables:

Bedroom AbvGr, Kitchen AbvGr, TotRms AbvGrd, Fireplaces, Garage Cars, Bsmt Full Bath, Bsmt Half Bath, Full Bath, Half Bath.

[Search] Analyzing Outliers in Continuous Variables Using Box Plot Method [Down]

[22] Python

```
import matplotlib.pyplot as plt
import seaborn as sns

continuous_cols = ['Lot Frontage', 'Lot Area', 'Mas Vnr Area', 'BsmtFin SF 1', 'BsmtFin SF 2', 'Bsmt Unf SF', 'Total Bsmt SF', '1st Flr SF', '2nd Flr SF', 'Gr Liv Area',
```



```
'Garage Area', 'Wood Deck SF', 'Open Porch SF', 'Enclosed Porch',
'Screen Porch', 'Pool Area']
for col in continuous_cols:
plt.figure(figsize=(8, 5))
sns.boxplot(y=df_dropped[col])
plt.title(f'Box Plot of {col}')
plt.ylabel(col)
plt.show()
```

Outliers identified through box plots

[23] Python

```
# Save the current DataFrame to a CSV file for later use
df_dropped.to_csv('cleaned_dataset_for_outliers.csv', index=False)
display("Current DataFrame saved to 'cleaned_dataset_for_outliers.csv'")
```

[24] Python

```
import pandas as pd
import numpy as np
```

[25] Python

```
# Load the dataset into a pandas DataFrame
df = pd.read_csv('cleaned_dataset_for_outliers.csv')
```

[Plant] Task: Now we are going handle Outliers in All Continuous Variables Accordingly

[26] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3)
Q1 = df['Lot Frontage'].quantile(0.25)
Q3 = df['Lot Frontage'].quantile(0.75)
# Calculate the Interquartile Range (IQR)
IQR = Q3 - Q1
# Define the lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
# Identify outliers
outliers = df[(df['Lot Frontage'] < lower_bound) | (df['Lot Frontage'] >
upper_bound)]
# Display the number of outliers and the outlier values
display(f"Number of outliers in 'Lot Frontage': {len(outliers)}")
display("Outliers in 'Lot Frontage':")
display(outliers['Lot Frontage'])
```

Decision on Handling Outliers in 'Lot Frontage'

- The detected outliers appear to be **genuine**, as the dataset includes **large properties** like **farmhouses** and **commercial plots**.
- Hence, removing them would **cause loss of valid data** and make the model **biased**.
- Instead, we will **cap these outliers** using the **upper limit of the IQR range** through the **capping method**.

- This ensures the **distribution remains balanced** while preserving genuine variability and preventing model distortion. [Down]

[Ruler] Capping Outliers in 'Lot Frontage' Using Upper Bound

Capping the extreme values in Lot Frontage at the calculated **upper bound** to limit high-end outliers and ensure a realistic maximum frontage value for properties. [Down]

[27] Python

```
# Cap the outliers in 'Lot Frontage' with the upper bound
df['Lot Frontage'] = np.where(df['Lot Frontage'] > upper_bound, upper_bound,
df['Lot Frontage'])

# Verify that the maximum value is now the upper bound (or lower)
display(f"Maximum value in 'Lot Frontage' after capping: {df['Lot
Frontage'].max()}")
```

[Ruler] Handle Outliers in 'Lot Area'

[Gear] Subtask:

- Identify and **cap outliers** in the Lot Area column using the **Interquartile Range (IQR)** method.

[Idea] Reasoning:

- Calculate **Q1**, **Q3**, and the **IQR (Interquartile Range)** for Lot Area.
- Determine the **lower** and **upper bounds** to detect extreme property sizes.
- **Cap the outliers** that fall outside these bounds to maintain data consistency and reduce the influence of large lot size variations.

[28] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3)
Q1_lot_area = df['Lot Area'].quantile(0.25)
Q3_lot_area = df['Lot Area'].quantile(0.75)

# Calculate the Interquartile Range (IQR)
IQR_lot_area = Q3_lot_area - Q1_lot_area

# Define the lower and upper bounds for outliers
lower_bound_lot_area = Q1_lot_area - 1.5 * IQR_lot_area
upper_bound_lot_area = Q3_lot_area + 1.5 * IQR_lot_area

# Cap the outliers in 'Lot Area' with the upper bound and lower bound
df['Lot Area'] = np.where(df['Lot Area'] > upper_bound_lot_area,
upper_bound_lot_area, df['Lot Area'])
df['Lot Area'] = np.where(df['Lot Area'] < lower_bound_lot_area,
lower_bound_lot_area, df['Lot Area'])

# Verify that the maximum value is now the upper bound (or lower)
display(f"Maximum value in 'Lot Area' after capping: {df['Lot Area'].max()}")
display(f"Minimum value in 'Lot Area' after capping: {df['Lot Area'].min()}")
```

[Brick] Handle Outliers in 'Mas Vnr Area'

[Gear] Subtask:

- Identify and **cap outliers** in the Mas Vnr Area column using the **Interquartile Range (IQR)** method.

[Idea] Reasoning:

- Calculate the **quartiles (Q1 and Q3)** and determine the **IQR (Interquartile Range)** for Mas Vnr Area.
- Compute the **lower** and **upper bounds** to identify extreme values.
- **Cap the outliers** lying beyond these limits to reduce skewness and stabilize the data distribution.
- Finally, **verify the results** by checking the **minimum and maximum values** after capping to ensure the range is realistic and consistent.

[29] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'Mas Vnr Area'
Q1_mas_vnr = df['Mas Vnr Area'].quantile(0.25)
Q3_mas_vnr = df['Mas Vnr Area'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'Mas Vnr Area'
IQR_mas_vnr = Q3_mas_vnr - Q1_mas_vnr

# Define the lower and upper bounds for outliers
lower_bound_mas_vnr = Q1_mas_vnr - 1.5 * IQR_mas_vnr
upper_bound_mas_vnr = Q3_mas_vnr + 1.5 * IQR_mas_vnr

# Cap the outliers in the 'Mas Vnr Area' column
df['Mas Vnr Area'] = np.where(df['Mas Vnr Area'] > upper_bound_mas_vnr,
                             upper_bound_mas_vnr, df['Mas Vnr Area'])
df['Mas Vnr Area'] = np.where(df['Mas Vnr Area'] < lower_bound_mas_vnr,
                             lower_bound_mas_vnr, df['Mas Vnr Area'])

# Display the maximum and minimum values of the 'Mas Vnr Area' column after capping
display(f"Maximum value in 'Mas Vnr Area' after capping: {df['Mas Vnr Area'].max()}")
display(f"Minimum value in 'Mas Vnr Area' after capping: {df['Mas Vnr Area'].min()}")
```

[Brick] Handle Outliers in 'BsmtFin SF 1'

[Gear] Subtask:

- Identify and **cap outliers** in the BsmtFin SF 1 column using the **Interquartile Range (IQR)** method.

[Idea] Reasoning:

- Calculate the **IQR**, determine the **lower** and **upper bounds**, and identify extreme values beyond these limits.
- **Cap the outliers** exceeding these bounds to reduce the effect of unusually large finished basement areas.
- This helps maintain a **realistic and balanced data distribution** for accurate modeling.

[30] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'BsmtFin SF 1'
Q1_bsmtfin1 = df['BsmtFin SF 1'].quantile(0.25)
Q3_bsmtfin1 = df['BsmtFin SF 1'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'BsmtFin SF 1'
IQR_bsmtfin1 = Q3_bsmtfin1 - Q1_bsmtfin1

# Define the lower and upper bounds for outliers
lower_bound_bsmtfin1 = Q1_bsmtfin1 - 1.5 * IQR_bsmtfin1
upper_bound_bsmtfin1 = Q3_bsmtfin1 + 1.5 * IQR_bsmtfin1

# Cap the outliers in the 'BsmtFin SF 1' column
```

```
df['BsmtFin SF 1'] = np.where(df['BsmtFin SF 1'] > upper_bound_bsmtfin1,
upper_bound_bsmtfin1, df['BsmtFin SF 1'])
df['BsmtFin SF 1'] = np.where(df['BsmtFin SF 1'] < lower_bound_bsmtfin1,
lower_bound_bsmtfin1, df['BsmtFin SF 1'])

# Display the maximum and minimum values of the 'BsmtFin SF 1' column after capping
display(f"Maximum value in 'BsmtFin SF 1' after capping: {df['BsmtFin SF
1'].max()}")
display(f"Minimum value in 'BsmtFin SF 1' after capping: {df['BsmtFin SF
1'].min()}")
```

[Brick] Handle Outliers in 'BsmtFin SF 2'

[Gear] Subtask:

- Identify and **cap outliers** in the BsmtFin SF 2 column using the **Interquartile Range (IQR)** method.
-

[Idea] Reasoning:

- Calculate **Q1**, **Q3**, and the **IQR (Interquartile Range)** for BsmtFin SF 2.
- Determine the **lower** and **upper bounds** to identify extreme values.
- **Cap the outliers** beyond these limits to reduce distortion caused by rare extreme basement areas.
- Finally, **verify the result** by checking the **minimum and maximum values** after capping to ensure realistic data distribution.

[31] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'BsmtFin SF 2'
Q1_bsmtfin2 = df['BsmtFin SF 2'].quantile(0.25)
Q3_bsmtfin2 = df['BsmtFin SF 2'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'BsmtFin SF 2'
IQR_bsmtfin2 = Q3_bsmtfin2 - Q1_bsmtfin2

# Define the lower and upper bounds for outliers
lower_bound_bsmtfin2 = Q1_bsmtfin2 - 1.5 * IQR_bsmtfin2
upper_bound_bsmtfin2 = Q3_bsmtfin2 + 1.5 * IQR_bsmtfin2

# Cap the outliers in the 'BsmtFin SF 2' column
df['BsmtFin SF 2'] = np.where(df['BsmtFin SF 2'] > upper_bound_bsmtfin2,
upper_bound_bsmtfin2, df['BsmtFin SF 2'])
df['BsmtFin SF 2'] = np.where(df['BsmtFin SF 2'] < lower_bound_bsmtfin2,
lower_bound_bsmtfin2, df['BsmtFin SF 2'])

# Display the maximum and minimum values of the 'BsmtFin SF 2' column after capping
display(f"Maximum value in 'BsmtFin SF 2' after capping: {df['BsmtFin SF
2'].max()}")
display(f"Minimum value in 'BsmtFin SF 2' after capping: {df['BsmtFin SF
2'].min()}")
```

[Brick] Handle Outliers in 'Bsmt Unf SF'

[Gear] Subtask:

- Identify and **cap outliers** in the Bsmt Unf SF column using the **Interquartile Range (IQR)** method.
-

[Idea] Reasoning:

- Calculate the **IQR**, determine the **lower** and **upper bounds**, and identify extreme values beyond these limits.

- **Cap the outliers** that exceed the acceptable range to reduce their influence on model training.
- This ensures that basement area values remain **statistically balanced** and **represent realistic property sizes**.

[32] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'Bsmt Unf SF'
Q1_bsmtunf = df['Bsmt Unf SF'].quantile(0.25)
Q3_bsmtunf = df['Bsmt Unf SF'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'Bsmt Unf SF'
IQR_bsmtunf = Q3_bsmtunf - Q1_bsmtunf

# Define the lower and upper bounds for outliers
lower_bound_bsmtunf = Q1_bsmtunf - 1.5 * IQR_bsmtunf
upper_bound_bsmtunf = Q3_bsmtunf + 1.5 * IQR_bsmtunf

# Cap the outliers in the 'Bsmt Unf SF' column
df['Bsmt Unf SF'] = np.where(df['Bsmt Unf SF'] > upper_bound_bsmtunf,
                             upper_bound_bsmtunf, df['Bsmt Unf SF'])
df['Bsmt Unf SF'] = np.where(df['Bsmt Unf SF'] < lower_bound_bsmtunf,
                             lower_bound_bsmtunf, df['Bsmt Unf SF'])

# Display the maximum and minimum values of the 'Bsmt Unf SF' column after capping
display(f"Maximum value in 'Bsmt Unf SF' after capping: {df['Bsmt Unf SF'].max()}")
display(f"Minimum value in 'Bsmt Unf SF' after capping: {df['Bsmt Unf SF'].min()}")
```

[Brick] Handle Outliers in 'Total Bsmt SF'

[Gear] Subtask:

- Identify and **cap outliers** in the Total Bsmt SF column using the **Interquartile Range (IQR)** method.

[Idea] Reasoning:

- Calculate **Q1**, **Q3**, and the **IQR (Interquartile Range)** for Total Bsmt SF.
- Determine the **lower** and **upper bounds** to identify potential outliers.
- **Cap the outliers** beyond these limits to minimize the influence of extreme basement sizes.
- Finally, **display the minimum and maximum values** after capping to confirm that the range is realistic and properly normalized.

[33] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'Total Bsmt SF'
Q1_totalbsmt = df['Total Bsmt SF'].quantile(0.25)
Q3_totalbsmt = df['Total Bsmt SF'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'Total Bsmt SF'
IQR_totalbsmt = Q3_totalbsmt - Q1_totalbsmt

# Define the lower and upper bounds for outliers
lower_bound_totalbsmt = Q1_totalbsmt - 1.5 * IQR_totalbsmt
upper_bound_totalbsmt = Q3_totalbsmt + 1.5 * IQR_totalbsmt

# Cap the outliers in the 'Total Bsmt SF' column
df['Total Bsmt SF'] = np.where(df['Total Bsmt SF'] > upper_bound_totalbsmt,
                             upper_bound_totalbsmt, df['Total Bsmt SF'])
df['Total Bsmt SF'] = np.where(df['Total Bsmt SF'] < lower_bound_totalbsmt,
                             lower_bound_totalbsmt, df['Total Bsmt SF'])

# Display the maximum and minimum values of the 'Total Bsmt SF' column after capping
```

```
display(f"Maximum value in 'Total Bsmt SF' after capping: {df['Total Bsmt SF'].max()}")
display(f"Minimum value in 'Total Bsmt SF' after capping: {df['Total Bsmt SF'].min()}")
```

[House] Handle Outliers in '1st Flr SF'

[Gear] Subtask:

- Identify and **cap outliers** in the 1st Flr SF column using the **Interquartile Range (IQR)** method.
-

[Idea] Reasoning:

- Calculate **Q1**, **Q3**, and the **IQR (Interquartile Range)** for 1st Flr SF.
- Determine the **lower** and **upper bounds** using these values.
- **Cap the outliers** that fall outside these bounds to limit the effect of extreme floor area values and maintain a balanced distribution.

[34] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for '1st Flr SF'
Q1_1stflrsf = df['1st Flr SF'].quantile(0.25)
Q3_1stflrsf = df['1st Flr SF'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for '1st Flr SF'
IQR_1stflrsf = Q3_1stflrsf - Q1_1stflrsf

# Define the lower and upper bounds for outliers
lower_bound_1stflrsf = Q1_1stflrsf - 1.5 * IQR_1stflrsf
upper_bound_1stflrsf = Q3_1stflrsf + 1.5 * IQR_1stflrsf

# Cap the outliers in the '1st Flr SF' column
df['1st Flr SF'] = np.where(df['1st Flr SF'] > upper_bound_1stflrsf,
                           upper_bound_1stflrsf, df['1st Flr SF'])
df['1st Flr SF'] = np.where(df['1st Flr SF'] < lower_bound_1stflrsf,
                           lower_bound_1stflrsf, df['1st Flr SF'])

# Display the maximum and minimum values of the '1st Flr SF' column after capping
display(f"Maximum value in '1st Flr SF' after capping: {df['1st Flr SF'].max()}")
display(f"Minimum value in '1st Flr SF' after capping: {df['1st Flr SF'].min()}")
```

[Office] Handle Outliers in '2nd Flr SF'

[Gear] Subtask:

- Identify and **cap outliers** in the 2nd Flr SF column using the **Interquartile Range (IQR)** method.
-

[Idea] Reasoning:

- Calculate **Q1**, **Q3**, and the **IQR (Interquartile Range)** for 2nd Flr SF.
- Determine the **lower** and **upper bounds** to identify extreme values.
- **Cap outliers** that fall outside these bounds to reduce distortion in the data.
- Finally, **display the minimum and maximum values** after capping to verify that the range is realistic and well-balanced.

[35] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for '2nd Flr SF'
```

```

Q1_2ndflrsf = df['2nd Flr SF'].quantile(0.25)
Q3_2ndflrsf = df['2nd Flr SF'].quantile(0.75)
# Calculate the Interquartile Range (IQR) for '2nd Flr SF'
IQR_2ndflrsf = Q3_2ndflrsf - Q1_2ndflrsf
# Define the lower and upper bounds for outliers
lower_bound_2ndflrsf = Q1_2ndflrsf - 1.5 * IQR_2ndflrsf
upper_bound_2ndflrsf = Q3_2ndflrsf + 1.5 * IQR_2ndflrsf
# Cap the outliers in the '2nd Flr SF' column
df['2nd Flr SF'] = np.where(df['2nd Flr SF'] > upper_bound_2ndflrsf,
upper_bound_2ndflrsf, df['2nd Flr SF'])
df['2nd Flr SF'] = np.where(df['2nd Flr SF'] < lower_bound_2ndflrsf,
lower_bound_2ndflrsf, df['2nd Flr SF'])
# Display the maximum and minimum values of the '2nd Flr SF' column after capping
display(f"Maximum value in '2nd Flr SF' after capping: {df['2nd Flr SF'].max()}")
display(f"Minimum value in '2nd Flr SF' after capping: {df['2nd Flr SF'].min()}")

```

[Home] Handle Outliers in 'Gr Liv Area'

[Gear] Subtask:

- Identify and **cap outliers** in the Gr Liv Area column using the **Interquartile Range (IQR)** method.
-

[Idea] Reasoning:

- Calculate **Q1**, **Q3**, and the **IQR** (Interquartile Range).
- Determine the **lower** and **upper bounds** for acceptable data values.
- **Cap the outliers** that fall outside these limits to reduce the impact of extreme values.
- Finally, **display the minimum and maximum values** after capping to confirm a realistic and balanced range.

[36] Python

```

# Calculate the first quartile (Q1) and third quartile (Q3) for 'Gr Liv Area'
Q1_gr_liv_area = df['Gr Liv Area'].quantile(0.25)
Q3_gr_liv_area = df['Gr Liv Area'].quantile(0.75)
# Calculate the Interquartile Range (IQR) for 'Gr Liv Area'
IQR_gr_liv_area = Q3_gr_liv_area - Q1_gr_liv_area
# Define the lower and upper bounds for outliers
lower_bound_gr_liv_area = Q1_gr_liv_area - 1.5 * IQR_gr_liv_area
upper_bound_gr_liv_area = Q3_gr_liv_area + 1.5 * IQR_gr_liv_area
# Cap the outliers in the 'Gr Liv Area' column
df['Gr Liv Area'] = np.where(df['Gr Liv Area'] > upper_bound_gr_liv_area,
upper_bound_gr_liv_area, df['Gr Liv Area'])
df['Gr Liv Area'] = np.where(df['Gr Liv Area'] < lower_bound_gr_liv_area,
lower_bound_gr_liv_area, df['Gr Liv Area'])
# Display the maximum and minimum values of the 'Gr Liv Area' column after capping
display(f"Maximum value in 'Gr Liv Area' after capping: {df['Gr Liv Area'].max()}")
display(f"Minimum value in 'Gr Liv Area' after capping: {df['Gr Liv Area'].min()}")

```

[Car] Handle Outliers in 'Garage Area'

[Gear] Subtask:

- Identify and **cap outliers** in the Garage Area column using the **Interquartile Range (IQR)** method.

[Idea] Reasoning:

- Calculate the **IQR**, determine the **lower and upper bounds**, and **cap outliers** that fall outside these limits.
- This helps control extremely large or small garage sizes that could skew the data.
- After capping, **display the minimum and maximum values** to verify that the range is now realistic and balanced.

[37] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'Garage Area'
Q1_garage_area = df['Garage Area'].quantile(0.25)
Q3_garage_area = df['Garage Area'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'Garage Area'
IQR_garage_area = Q3_garage_area - Q1_garage_area

# Define the lower and upper bounds for outliers
lower_bound_garage_area = Q1_garage_area - 1.5 * IQR_garage_area
upper_bound_garage_area = Q3_garage_area + 1.5 * IQR_garage_area

# Cap the outliers in the 'Garage Area' column
df['Garage Area'] = np.where(df['Garage Area'] > upper_bound_garage_area,
                             upper_bound_garage_area, df['Garage Area'])
df['Garage Area'] = np.where(df['Garage Area'] < lower_bound_garage_area,
                             lower_bound_garage_area, df['Garage Area'])

# Display the maximum and minimum values of the 'Garage Area' column after capping
display(f"Maximum value in 'Garage Area' after capping: {df['Garage Area'].max()}")
display(f"Minimum value in 'Garage Area' after capping: {df['Garage Area'].min()}")
```

[Wood] Handle Outliers in 'Wood Deck SF'

[Gear] Subtask:

- Identify and **cap outliers** in the wood Deck SF column using the **Interquartile Range (IQR)** method.
-

[Idea] Reasoning:

- Calculate the **IQR**, determine the **lower and upper bounds**, and **cap outliers** beyond these limits.
- This ensures that **extreme wood deck sizes** do not distort the data distribution.
- The step maintains a **balanced range** of values, preserving genuine property variations while reducing model bias.

[38] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'Wood Deck SF'
Q1_wood_deck_sf = df['Wood Deck SF'].quantile(0.25)
Q3_wood_deck_sf = df['Wood Deck SF'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'Wood Deck SF'
IQR_wood_deck_sf = Q3_wood_deck_sf - Q1_wood_deck_sf

# Define the lower and upper bounds for outliers
lower_bound_wood_deck_sf = Q1_wood_deck_sf - 1.5 * IQR_wood_deck_sf
upper_bound_wood_deck_sf = Q3_wood_deck_sf + 1.5 * IQR_wood_deck_sf

# Cap the outliers in the 'Wood Deck SF' column
df['Wood Deck SF'] = np.where(df['Wood Deck SF'] > upper_bound_wood_deck_sf,
                              upper_bound_wood_deck_sf, df['Wood Deck SF'])
df['Wood Deck SF'] = np.where(df['Wood Deck SF'] < lower_bound_wood_deck_sf,
                              lower_bound_wood_deck_sf, df['Wood Deck SF'])
```



```
# Display the maximum and minimum values of the 'Wood Deck SF' column after capping
display(f"Maximum value in 'Wood Deck SF' after capping: {df['Wood Deck SF'].max()}")

display(f"Minimum value in 'Wood Deck SF' after capping: {df['Wood Deck SF'].min()}")
```

[Plant] Handle Outliers in 'Open Porch SF'

[Gear] Subtask:

- Identify and **cap outliers** in the Open Porch SF column using the **Interquartile Range (IQR)** method.

[Idea] Reasoning:

- Calculate the **IQR**, determine the **lower and upper bounds**, and **cap outliers** beyond these limits.
- After capping, **verify the minimum and maximum values** to ensure all data points fall within a realistic range.
- This process maintains the **natural variation** of porch sizes while preventing extreme values from biasing the model.

[39] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'Open Porch SF'
Q1_open_porch_sf = df['Open Porch SF'].quantile(0.25)
Q3_open_porch_sf = df['Open Porch SF'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'Open Porch SF'
IQR_open_porch_sf = Q3_open_porch_sf - Q1_open_porch_sf

# Define the lower and upper bounds for outliers
lower_bound_open_porch_sf = Q1_open_porch_sf - 1.5 * IQR_open_porch_sf
upper_bound_open_porch_sf = Q3_open_porch_sf + 1.5 * IQR_open_porch_sf

# Cap the outliers in the 'Open Porch SF' column
df['Open Porch SF'] = np.where(df['Open Porch SF'] > upper_bound_open_porch_sf,
                               upper_bound_open_porch_sf, df['Open Porch SF'])
df['Open Porch SF'] = np.where(df['Open Porch SF'] < lower_bound_open_porch_sf,
                               lower_bound_open_porch_sf, df['Open Porch SF'])

# Display the maximum and minimum values of the 'Open Porch SF' column after capping
display(f"Maximum value in 'Open Porch SF' after capping: {df['Open Porch SF'].max()}")
display(f"Minimum value in 'Open Porch SF' after capping: {df['Open Porch SF'].min()}")
```

[House] Handle Outliers in 'Enclosed Porch'

[Gear] Subtask:

- Identify and **cap outliers** in the Enclosed Porch column using the **Interquartile Range (IQR)** method.

[Idea] Reasoning:

- Calculate the **IQR**, **lower bound**, and **upper bound** for Enclosed Porch.
- **Cap values** lying beyond these limits to reduce the influence of extreme values.
- This ensures a **more uniform distribution** and prevents rare anomalies from affecting model training.

[40] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'Enclosed Porch'
Q1_enclosed_porch = df['Enclosed Porch'].quantile(0.25)
Q3_enclosed_porch = df['Enclosed Porch'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'Enclosed Porch'
IQR_enclosed_porch = Q3_enclosed_porch - Q1_enclosed_porch

# Define the lower and upper bounds for outliers
lower_bound_enclosed_porch = Q1_enclosed_porch - 1.5 * IQR_enclosed_porch
upper_bound_enclosed_porch = Q3_enclosed_porch + 1.5 * IQR_enclosed_porch

# Cap the outliers in the 'Enclosed Porch' column
df['Enclosed Porch'] = np.where(df['Enclosed Porch'] > upper_bound_enclosed_porch,
                                upper_bound_enclosed_porch, df['Enclosed Porch'])
df['Enclosed Porch'] = np.where(df['Enclosed Porch'] < lower_bound_enclosed_porch,
                                lower_bound_enclosed_porch, df['Enclosed Porch'])

# Display the maximum and minimum values of the 'Enclosed Porch' column after capping
display(f"Maximum value in 'Enclosed Porch' after capping: {df['Enclosed Porch'].max()}")
display(f"Minimum value in 'Enclosed Porch' after capping: {df['Enclosed Porch'].min()}")
```

[Home] Handle Outliers in 'Screen Porch'

[Gear] Subtask:

- Identify and **cap outliers** in the Screen Porch column using the **Interquartile Range (IQR)** method.

[Idea] Reasoning:

- Calculate the **IQR**, **lower bound**, and **upper bound** for Screen Porch.
- **Cap the outliers** beyond these bounds to limit extreme deviations.
- This ensures a **balanced distribution** and prevents rare extreme values from distorting the model.

[41] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'Screen Porch'
Q1_screen_porch = df['Screen Porch'].quantile(0.25)
Q3_screen_porch = df['Screen Porch'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'Screen Porch'
IQR_screen_porch = Q3_screen_porch - Q1_screen_porch

# Define the lower and upper bounds for outliers
lower_bound_screen_porch = Q1_screen_porch - 1.5 * IQR_screen_porch
upper_bound_screen_porch = Q3_screen_porch + 1.5 * IQR_screen_porch

# Cap the outliers in the 'Screen Porch' column
df['Screen Porch'] = np.where(df['Screen Porch'] > upper_bound_screen_porch,
                                upper_bound_screen_porch, df['Screen Porch'])
df['Screen Porch'] = np.where(df['Screen Porch'] < lower_bound_screen_porch,
                                lower_bound_screen_porch, df['Screen Porch'])

# Display the maximum and minimum values of the 'Screen Porch' column after capping
display(f"Maximum value in 'Screen Porch' after capping: {df['Screen Porch'].max()}")
display(f"Minimum value in 'Screen Porch' after capping: {df['Screen Porch'].min()}")
```

[Swim]■ Handle Outliers in 'Pool Area'

[Gear] Subtask:

- Identify and **cap outliers** in the Pool Area column using the **Interquartile Range (IQR)** method.

[Idea] Reasoning:

- Calculate the **IQR**, **lower bound**, and **upper bound** for Pool Area.
- Cap values that fall **outside these bounds** to limit extreme deviations.
- Finally, **display the minimum and maximum values** after capping

to verify that the column has been successfully normalized.

[42] Python

```
# Calculate the first quartile (Q1) and third quartile (Q3) for 'Pool Area'
Q1_pool_area = df['Pool Area'].quantile(0.25)
Q3_pool_area = df['Pool Area'].quantile(0.75)

# Calculate the Interquartile Range (IQR) for 'Pool Area'
IQR_pool_area = Q3_pool_area - Q1_pool_area

# Define the lower and upper bounds for outliers
lower_bound_pool_area = Q1_pool_area - 1.5 * IQR_pool_area
upper_bound_pool_area = Q3_pool_area + 1.5 * IQR_pool_area

# Cap the outliers in the 'Pool Area' column
df['Pool Area'] = np.where(df['Pool Area'] > upper_bound_pool_area,
                           upper_bound_pool_area, df['Pool Area'])
df['Pool Area'] = np.where(df['Pool Area'] < lower_bound_pool_area,
                           lower_bound_pool_area, df['Pool Area'])

# Display the maximum and minimum values of the 'Pool Area' column after capping
display(f"Maximum value in 'Pool Area' after capping: {df['Pool Area'].max()}")
display(f"Minimum value in 'Pool Area' after capping: {df['Pool Area'].min()}")
```

[Cut] Removing Extremely Sparse Features

- Features like BsmtFin SF 2, Enclosed Porch, Screen Porch, and Pool Area may seem relevant for price prediction.
- However, their **non-zero values were less than 0.1%**, making them **extremely sparse** in the dataset.
- Such sparse features add **minimal predictive value** and can **negatively impact model performance**.
- Hence, it is **better to remove** these columns to maintain a clean and efficient dataset for modeling. [OK]

[43] Python

```
# List of columns to drop due to high sparsity
columns_to_drop_sparse = ['Enclosed Porch', 'Screen Porch', 'Pool Area']

# Drop the columns from the DataFrame
df_dropped = df.drop(columns=columns_to_drop_sparse)

# Display the first 5 rows of the updated DataFrame to confirm removal
display(df_dropped.head())
```

[Chart] Checking Updated Distributions After Outlier Handling

After handling all outliers in **continuous variables**,

we will now **check the updated distributions** of these variables using **histograms**

to visually confirm normalization and overall data balance. [\[Down\]](#)

[44] Python

```
import matplotlib.pyplot as plt
import seaborn as sns

# List of remaining continuous numerical columns after dropping sparse ones
continuous_cols_remaining = ['Lot Frontage', 'Lot Area', 'Mas Vnr Area', 'BsmtFin
SF 1', 'BsmtFin SF 2',
'Bsmt Unf SF', 'Total Bsmt SF', '1st Flr SF', '2nd Flr SF', 'Gr Liv Area',
'Garage Area', 'Wood Deck SF', 'Open Porch SF']

# Create distribution plots for each remaining continuous column
for col in continuous_cols_remaining:
    plt.figure(figsize=(8, 5))
    sns.histplot(df_dropped[col], kde=True)
    plt.title(f'Distribution of {col} (Capped)')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.show()
```

[45] Python

```
# Save the DataFrame to a CSV file
df_dropped.to_csv('cleaned_dataset_capped.csv', index=False)
display("DataFrame saved to 'cleaned_dataset_capped.csv'")
```

[\[Search\]](#) Now Analyzing Discrete Variables Using Box Plot Method [\[Down\]](#)

We will now analyze the following **discrete variables** using the **box plot method** to visually detect **outliers** and understand their **value distributions**:

Bedroom AbvGr, Kitchen AbvGr, TotRms AbvGrd, Fireplaces,
Garage Cars, Bsmt Full Bath, Bsmt Half Bath, Full Bath, Half Bath.

[46] Python

```
import matplotlib.pyplot as plt
import seaborn as sns

discrete_cols = ['Bedroom AbvGr', 'Kitchen AbvGr', 'TotRms AbvGrd', 'Fireplaces',
'Garage Cars', 'Bsmt Full Bath', 'Bsmt Half Bath', 'Full Bath', 'Half Bath']

for col in discrete_cols:
    plt.figure(figsize=(8, 5))
    sns.boxplot(y=df_dropped[col]) # Use df_dropped to reflect previous cleaning
    plt.title(f'Box Plot of {col}')
    plt.ylabel(col)
    plt.show()
```

Outliers identified after seeing in box plot

[\[Tool\]](#)[\[Idea\]](#) Now handling the outliers accordingly in discrete variables [\[Down\]](#)

[47] Python

```
# Calculate the median of 'Bedroom AbvGr'
bedroom_median = df_dropped['Bedroom AbvGr'].median()

# Replace 0 values in 'Bedroom AbvGr' with the median
```

```
df_dropped['Bedroom AbvGr'] = df_dropped['Bedroom AbvGr'].replace(0,
bedroom_median)

# Verify the change by checking the value counts or minimum value
display("Value counts after replacing 0s:")
display(df_dropped['Bedroom AbvGr'].value_counts().sort_index())
display(f"Minimum value in 'Bedroom AbvGr' after replacement: {df_dropped['Bedroom
AbvGr'].min()}")
```

[Bed] Extreme Outlier in Bedroom Count

- A single record with **8 bedrooms** was identified as an **extreme structural outlier** (only one occurrence).
- Such an unusually large value could **bias the model** and reduce generalization capability.
- Hence, this record will be **removed** to maintain model stability and data reliability. [OK]

[48] Python

```
# Remove rows where 'Bedroom AbvGr' is 8
df_dropped = df_dropped[df_dropped['Bedroom AbvGr'] != 8]

# Verify the removal by checking the value counts or the maximum value
display("Value counts after removing row(s) with 8 bedrooms:")
display(df_dropped['Bedroom AbvGr'].value_counts().sort_index())
display(f"Maximum value in 'Bedroom AbvGr' after removal: {df_dropped['Bedroom
AbvGr'].max()}")
```

[Food] KitchenAbvGr Variable Analysis

- KitchenAbvGr is a **categorical-discrete variable**.
- Most homes have **1 kitchen**, while **2–3 kitchens** represent **genuine outliers** found in luxury or multi-unit houses.
- Records with **0 kitchens** are **invalid** and will be **corrected by replacing them with 1** to maintain logical consistency. [OK]

[49] Python

```
# Calculate the median of 'Kitchen AbvGr'
kitchen_median = df_dropped['Kitchen AbvGr'].median()

# Replace 0 values in 'Kitchen AbvGr' with the median
df_dropped['Kitchen AbvGr'] = df_dropped['Kitchen AbvGr'].replace(0,
kitchen_median)

# Verify the change by checking the value counts or minimum value
display("Value counts after replacing 0s:")
display(df_dropped['Kitchen AbvGr'].value_counts().sort_index())
display(f"Minimum value in 'Kitchen AbvGr' after replacement: {df_dropped['Kitchen
AbvGr'].min()}")
```

[Warn] Unrealistic Record Identified. Going for Removal:

- A record with TotRmsAbvGrd = 2 and GrLivArea = 334 sq.ft has been identified as **unrealistic**.
- Such extremely low values are likely due to a **data entry error** or may represent a **non-residential structure**.
- Hence, we are **removing this row** to ensure data accuracy and maintain dataset integrity. [OK]

[50] Python

```
df[df['TotRms AbvGrd'] <= 2][['TotRms AbvGrd', 'Gr Liv Area', 'Bedroom AbvGr',
'Full Bath']]
```

[51] Python

```
# Remove the row where 'TotRms AbvGrd' is 2 and 'Gr Liv Area' is 334.0
df_dropped = df_dropped[~((df_dropped['TotRms AbvGrd'] == 2) & (df_dropped['Gr Liv Area'] == 334.0))]

# Verify the removal by checking the row with the specific criteria
display("Checking if the row with TotRms AbvGrd = 2 and Gr Liv Area = 334.0 still exists:")
display(df_dropped[(df_dropped['TotRms AbvGrd'] == 2) & (df_dropped['Gr Liv Area'] == 334.0)])
```

[Receipt] Outlier Handling summary in Discrete Variables:

- All remaining **discrete variables** (Fireplaces, Garage Cars, Bsmt Full Bath, Bsmt Half Bath, Full Bath, Half Bath) were **retained as-is**.
- Their high-end outliers represent **genuine luxury properties**, not data errors.
- These extreme but valid values capture **important variations in housing features** and are valuable for accurate price prediction.

[Check] The process of handling outliers in discrete variables has been successfully completed.

[Search] Checking for the Following Points and Fixing Them :

###- **Low-frequency columns** (rare categories with very few occurrences)

####- **Irrelevant attributes** that do not contribute to modeling

####- **Spelling inconsistencies** in categorical column names and values

#####[Down] Proceeding with detailed inspection and corrections.

[52] Python

```
import matplotlib.pyplot as plt
import seaborn as sns

# Select only categorical columns from the DataFrame
categorical_cols = df_dropped.select_dtypes(include='object').columns

# Create a countplot for each categorical column
for col in categorical_cols:
    plt.figure(figsize=(10, 6))
    sns.countplot(data=df_dropped, y=col, order=df_dropped[col].value_counts().index,
                  palette='viridis', hue=col, legend=False)
    plt.title(f'Countplot of {col}')
    plt.xlabel('Count')
    plt.ylabel(col)
    plt.tight_layout()
    plt.show()
```

[53] Python

```
# Save the DataFrame to a CSV file
df_dropped.to_csv('cleaned_dataset_latest.csv', index=False)

display("DataFrame saved to 'cleaned_dataset_latest.csv'")
```

[54] Python

```
import pandas as pd

# Load the cleaned dataset
```

```
df_dropped = pd.read_csv('cleaned_dataset_latest.csv')

# Display the first few rows to confirm it's loaded
display(df_dropped.head())
```

[Calc] Identifying Low-Frequency Categories Using Code[Down]

[55] Python

```
# List of categorical columns identified as having low-frequency categories
low_frequency_cols = ['MS Zoning', 'Street', 'Utilities', 'Land Slope', 'Condition
1',
'Condition 2', 'Roof Matl', 'Exterior 1st', 'Exterior 2nd',
'Heating', 'Electrical', 'Functional', 'Fireplace Qu',
'Garage Type', 'Garage Finish', 'Garage Qual', 'Garage Cond',
'Paved Drive', 'Sale Type', 'Sale Condition']

for col in low_frequency_cols:
display(f"Value counts for '{col}':")
display(df_dropped[col].value_counts())
display("-" * 30) # Separator for readability
```

Columns identified now going to apply grouping [Down]

[56] Python

```
# List of categorical columns to apply grouping to
categorical_cols_to_group = ['MS Zoning', 'Street', 'Utilities', 'Land Slope',
'Condition 1',
'Condition 2', 'Roof Matl', 'Exterior 1st', 'Exterior 2nd',
'Heating', 'Electrical', 'Functional', 'Fireplace Qu',
'Garage Type', 'Garage Finish', 'Garage Qual', 'Garage Cond',
'Paved Drive', 'Sale Type', 'Sale Condition']

# Define the threshold for rare categories (1%)
threshold = 0.01

for col in categorical_cols_to_group:
# Calculate the frequency of each category
freq = df_dropped[col].value_counts(normalize=True)

# Identify rare categories (frequency less than the threshold)
rare_categories = freq[freq < threshold].index

# Replace rare categories with 'Others'
df_dropped[col] = df_dropped[col].replace(rare_categories, 'Others')

# Display value counts after grouping to verify
display(f"Value counts for '{col}' after grouping rare categories:")
display(df_dropped[col].value_counts())
display("-" * 30) # Separator for readability
```

[Check] Rare Category Grouping Summary

- Rare categories (with frequency < 1%) in categorical variables have been **grouped under the label 'Others'** using automated frequency-based encoding.
- This step ensures **data consistency** across all categorical columns.
- Helps **prevent model bias** due to sparse or low-frequency categories.
- **Preserves categorical diversity** while minimizing unnecessary noise.
- Final categorical columns now have **balanced distributions**, ready for **encoding and modeling**.

[Search] Checking for Irrelevant Columns:

Inspecting columns such as order and PID to identify and remove any non-informative identifiers.

[57] Python

```
# Identify columns that might be irrelevant (like IDs)
irrelevant_cols = ['Order', 'PID'] # Assuming 'Order' and 'PID' are identifiers

# Identify columns with only one unique value
single_category_cols = [col for col in df_dropped.columns if
df_dropped[col].nunique() == 1]

# Combine the lists
columns_to_consider_dropping = irrelevant_cols + single_category_cols

display("Columns to consider dropping (Irrelevant or Single Category):")
display(columns_to_consider_dropping)
```

[58] Python

```
display(df_dropped['BsmtFin SF 2'].unique())
```

[Cut] **Irrelevant identifier columns** (Order, PID) and the **zero-variance column** (BsmtFin SF 2) will be **removed from the dataset** to eliminate non-informative features and **enhance model efficiency**.

[59] Python

```
# List of columns to drop
columns_to_drop = ['Order', 'PID', 'BsmtFin SF 2']

# Drop the columns from the DataFrame
df_dropped = df_dropped.drop(columns=columns_to_drop)

# Display the head of the DataFrame and its shape to verify the columns are dropped
display(df_dropped.head())
display(f"Shape of the DataFrame after dropping columns: {df_dropped.shape}")
```

Checking spelling inconsistency

[60] Python

```
# Select only categorical columns from the DataFrame
categorical_cols = df_dropped.select_dtypes(include='object').columns

# Display unique values for each categorical column
for col in categorical_cols:
display(f"Unique values for '{col}':")
display(df_dropped[col].unique())
display("-" * 30) # Separator for readability
```

[61] Python

```
# Correct spelling inconsistency in 'Exterior 2nd'
df_dropped['Exterior 2nd'] = df_dropped['Exterior 2nd'].replace('CmentBd',
'CemntBd')

# Correct typo in 'Garage Type'
df_dropped['Garage Type'] = df_dropped['Garage Type'].replace('Basment',
'Basement')

# Remove leading/trailing spaces in 'Sale Type'
df_dropped['Sale Type'] = df_dropped['Sale Type'].str.strip()

# Optional: Verify the changes by checking unique values in these columns again
display("Unique values for 'Exterior 2nd' after correction:")
display(df_dropped['Exterior 2nd'].unique())
display("-" * 30)
display("Unique values for 'Garage Type' after correction:")
```



```
display(df_dropped['Garage Type'].unique())
display("-" * 30)
display("Unique values for 'Sale Type' after correction:")
display(df_dropped['Sale Type'].unique())
```

[OK] After correcting spelling inconsistencies and standardizing category names, all categorical columns now contain **clean, consistent, and verified unique values**.

This ensures **uniform labeling** across features like Exterior 2nd, Garage Type, and Sale Type, improving data quality for modeling.

[Warn] Unexpected Aspects Observed During Verification [Search]

It was observed that, during **dataset reloading**, the previously **imputed categorical values ('None')** in structural columns — Bsmt Qual, Bsmt Cond, Fireplace Qu, Garage Type, Garage Qual, and Garage Cond — were **temporarily lost** due to the reload process.

[62] Python

```
df_dropped[['Bsmt Qual', 'Bsmt Cond', 'Fireplace Qu', 'Garage Type', 'Garage Qual',
'Garage Cond']].isnull().sum()
```

[63] Python

```
# List of categorical columns with missing values to fill with 'None'
categorical_cols_to_fill_none = ['Bsmt Qual', 'Bsmt Cond', 'Fireplace Qu', 'Garage
Type', 'Garage Qual', 'Garage Cond']
# Fill missing values in these columns with 'None'
for col in categorical_cols_to_fill_none:
df_dropped[col] = df_dropped[col].fillna('None')
# Verify that there are no more missing values in these columns
display(df_dropped[categorical_cols_to_fill_none].isnull().sum())
```

[Tool][Check] Dataset Reload Correction Note

During dataset reloading, the previously **imputed categorical values ('None')** in key structural columns — Bsmt Qual, Bsmt Cond, Fireplace Qu, Garage Type, Garage Qual, and Garage Cond — were **temporarily lost** due to the reload process.

Upon verification, these missing entries were **re-filled with the label 'None'** to correctly represent the **absence of respective structural features** (e.g., no basement, no garage, or no fireplace).

[OK] This step ensures **full consistency with the original preprocessing logic** and maintains **structural integrity** across the entire dataset.

[Plant] Data Cleaning Successfully Completed

The dataset is now **pristine, consistent, and fully ready** for encoding and subsequent modeling.

[Receipt] Data Cleaning Completion Summary

The dataset has been successfully cleaned and prepared for modeling.

All preprocessing steps have been completed, including:

- [OK] **Handling of missing values** in both numeric and categorical variables.
- [OK] **Detection and capping of outliers** in continuous features using the **IQR method**.
- [OK] **Identification and grouping of rare categories (<1%)** under the label '**Others**' in categorical attributes.
- [OK] **Correction of inconsistent spellings, trailing spaces, and typographical errors**

(e.g., 'Basment' 'Basement', 'WD ' 'WD').

- [OK] **Logical replacement of structural missing values** with '**None**' to represent the absence of basement, garage, or fireplace features.
- [OK] **Removal of irrelevant and zero-variance columns** (Order, PID, BsmtFin SF 2) to eliminate non-informative data.
- [OK] **Verification of value consistency and frequency distributions** across all categorical features.

After completing these steps, the dataset now contains only **relevant, standardized, and fully cleaned features**

ready for the **further stages** ahead.

[64] Python

```
# Save the DataFrame to a CSV file
df_dropped.to_csv('fully_cleaned_dataset.csv', index=False)
display("DataFrame saved to 'fully_cleaned_dataset.csv'")
```

[65] Python

```
import pandas as pd
# Load the cleaned dataset
df_dropped = pd.read_csv('fully_cleaned_dataset.csv')
# Display the first few rows to confirm it's loaded
display(df_dropped.head())
```

[Green] Now Proceeding to Transform Continuous Variables

We will now apply appropriate **transformations** (such as *log1p* and *Yeo-Johnson*) to **continuous numerical features** in order to **reduce skewness**, improve **normality**, and enhance **model performance**.

[Target] Reason for Continuous Variable Transformation

- Many **continuous numerical variables** (e.g., Lot Area, Gr Liv Area, Total Bsmt SF, Mas Vnr Area) were **highly skewed**, deviating from the normal (bell-shaped) distribution.
- **Skewed distributions** can cause **bias** in regression and statistical models, as large values dominate the learning process.
- Such imbalance leads to **unstable model coefficients**, **poor generalization**, and reduced **prediction accuracy**.
- To correct this, **log1p** and **Yeo-Johnson** transformations were applied to normalize feature distributions.
- **log1p** handled **positively skewed** features, while **Yeo-Johnson** worked for features with **zero or negative values**.
- Post-transformation, continuous features became **statistically balanced** and **closer to normal distribution**.
- This process will help ensure **fair, stable, and efficient learning** for regression and machine learning models.

Just starting with one variable initially to check how it will perform

[66] Python

```
# Apply a log1p transformation to 'Lot Frontage' (log(1+x)) to handle zero values
df_dropped['Lot Frontage_log'] = np.log1p(df_dropped['Lot Frontage'])

# Display the first few rows to show the new column
display(df_dropped[['Lot Frontage', 'Lot Frontage_log']].head())

# Optionally, visualize the distribution of the transformed column
plt.figure(figsize=(8, 5))
sns.histplot(df_dropped['Lot Frontage_log'], kde=True)
plt.title('Distribution of Log-Transformed Lot Frontage')
plt.xlabel('Lot Frontage (Log Transformed)')
plt.ylabel('Frequency')
plt.show()
```

[Chart] Plotting Histograms of Numerical Columns [Down]

[Target] Reason for Plotting All Numerical Columns

- We are plotting histograms of **all numerical columns** — both **continuous** and **discrete**.
- To **visually analyze their distributions** and detect any **skewness or outliers**.
- To **clearly distinguish** between **continuous** and **discrete** variables.
- This helps in selecting the **appropriate transformation or scaling method** in later stages.

[67] Python

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Get a list of all numerical columns
numerical_cols = df_dropped.select_dtypes(include=np.number).columns.tolist()

# Separate continuous and discrete columns based on our previous analysis
continuous_cols = ['Lot Frontage', 'Lot Area', 'Mas Vnr Area', 'BsmtFin SF 1',
                   'Bsmt Unf SF', 'Total Bsmt SF', '1st Flr SF', '2nd Flr SF',
                   'Gr Liv Area', 'Garage Area', 'Wood Deck SF', 'Open Porch SF',
                   'Lot Frontage_log'] # Include the log-transformed Lot Frontage
discrete_cols = ['MS SubClass', 'Overall Qual', 'Overall Cond', 'Year Built',
                 'Year Remod/Add', 'Bsmt Full Bath', 'Bsmt Half Bath', 'Full Bath',
                 'Half Bath', 'Bedroom AbvGr', 'Kitchen AbvGr', 'TotRms AbvGrd',
                 'Fireplaces', 'Garage Yr Blt', 'Garage Cars', 'Misc Val', 'Mo Sold',
                 'Yr Sold', 'SalePrice'] # Include columns that are discrete or time-based

# Plot distributions for continuous variables
print("Plotting distributions for continuous variables...")
for col in continuous_cols:
    plt.figure(figsize=(8, 5))
    sns.histplot(df_dropped[col], kde=True)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')
    plt.show()

# Plot distributions for discrete variables
print("Plotting distributions for discrete variables...")
for col in discrete_cols:
    plt.figure(figsize=(8, 5))
    sns.countplot(data=df_dropped, x=col, palette='viridis', hue=col, legend=False)
    plt.title(f'Distribution of {col}')
    plt.xlabel(col)
```

```
plt.ylabel('Count')
plt.show()
```

[Puzzle] Task: Identify and Transform Skewed Continuous Variables

[Target] Objective:

Identify **skewed continuous variables** in the `df_dropped` DataFrame, apply appropriate **transformations** to reduce skewness, and verify the results.

[Gear] Subtask 1: Identify Skewed Continuous Variables

- **Action:** Calculate the **skewness** for each continuous numerical column.
 - **Goal:** Identify those columns whose **skewness exceeds a defined threshold** (e.g., $|\text{skew}| > 0.5$ or 1.0).
 - **Process:**
 - Select continuous numerical columns.
 - Compute their skewness values.
 - Set a skewness threshold.
 - Filter and list columns exceeding that limit.
-

[Idea] Reasoning:

- The subtask aims to **quantify the level of asymmetry** in each continuous variable's distribution.
- Detecting **highly skewed columns** helps in identifying which features need **normalization or transformation**.
- Reducing skewness leads to **more stable model coefficients** and **better predictive performance** in regression-based models.

```
[68] Python
```

```
# Select only the continuous numerical columns
continuous_cols_skew = df_dropped[continuous_cols]

# Calculate the skewness for each continuous column
skewness = continuous_cols_skew.skew()

# Define a skewness threshold
skewness_threshold = 0.5

# Identify columns with skewness above the threshold (absolute value)
highly_skewed_cols = skewness[abs(skewness) > skewness_threshold].index.tolist()

# Display the highly skewed columns
display("Highly skewed continuous columns (absolute skewness > 0.5):")
display(highly_skewed_cols)
```

[Idea] Finding and Further Process:

- The previous step **identified highly skewed continuous columns**.
- The next step is to **apply suitable transformations** (e.g., $\log_{1p}$, Yeo-Johnson) to these columns to **reduce skewness**.
- After transformation, we will **recalculate and display skewness values** to verify the improvement in distribution symmetry.
- This completes the **remaining part of the subtask**, ensuring that continuous variables are now **statistically normalized and ready for scaling**.

[69] Python

```
from sklearn.preprocessing import PowerTransformer

# Create a dictionary to store skewness values before and after transformation
skewness_comparison = {}

# Iterate through the highly skewed columns
for col in highly_skewed_cols:
    original_skewness = df_dropped[col].skew()

    # Check if the column has a significant number of zeros (e.g., more than 10%)
    zero_percentage = (df_dropped[col] == 0).sum() / len(df_dropped)

    if zero_percentage > 0.1: # Using 10% as a threshold for "many zeros"
        # Apply Yeo-Johnson transformation
        pt = PowerTransformer(method='yeo-johnson')
        # Reshape the data as PowerTransformer expects a 2D array
        df_dropped[f'{col}_yeojohnson'] = pt.fit_transform(df_dropped[[col]])
        transformed_skewness = df_dropped[f'{col}_yeojohnson'].skew()
        transformation_method = 'Yeo-Johnson'
    else:
        # Apply log1p transformation
        df_dropped[f'{col}_log1p'] = np.log1p(df_dropped[col])
        transformed_skewness = df_dropped[f'{col}_log1p'].skew()
        transformation_method = 'log1p'

    # Store the skewness values in the comparison dictionary
    skewness_comparison[col] = {
        'Original Skewness': original_skewness,
        'Transformed Skewness': transformed_skewness,
        'Transformation Method': transformation_method
    }

# Display the skewness comparison table
display("Skewness comparison before and after transformation:")
display(pd.DataFrame.from_dict(skewness_comparison, orient='index'))
```

[Idea] Findings and Next Step

- Most transformations worked perfectly — features like Mas Vnr Area, BsmtFin SF 1, 1st Flr SF, 2nd Flr SF, Gr Liv Area, Wood Deck SF, and Open Porch SF are now well-normalized and close to normal distribution.
- However, Bsmt Unf SF and Lot Frontage_log became **left-skewed** after log1p transformation.
- To fix this, we will apply **inverse (power-based) transformations** and re-evaluate their skewness to achieve a more symmetric distribution.

[70] Python

```
# Apply an inverse transformation (squaring) to 'Bsmt Unf SF'
df_dropped['Bsmt Unf SF_inverse'] = df_dropped['Bsmt Unf SF']**2

# Apply an exponential transformation to 'Lot Frontage_log'
df_dropped['Lot Frontage_log_inverse'] = np.exp(df_dropped['Lot Frontage_log'])

# Display the first few rows to show the new columns
display(df_dropped[['Bsmt Unf SF', 'Bsmt Unf SF_inverse', 'Lot Frontage_log', 'Lot Frontage_log_inverse']].head())
```

[71] Python

```
# --- Finalize transformations for 'Bsmt Unf SF' and 'Lot Frontage_log' ---

# Keep the inverse transformed 'Bsmt Unf SF' and drop the original and log1p transformed versions
```

```

# Note: We need to handle the case where the original 'Bsmt Unf SF' and 'Bsmt Unf
SF_log1p' might still be in the DataFrame
if 'Bsmt Unf SF' in df_dropped.columns:
df_dropped = df_dropped.drop(columns=['Bsmt Unf SF'], errors='ignore')
if 'Bsmt Unf SF_log1p' in df_dropped.columns:
df_dropped = df_dropped.drop(columns=['Bsmt Unf SF_log1p'], errors='ignore')
if 'Bsmt Unf SF_yeojohnson' in df_dropped.columns: # Also check for Yeo-Johnson in
case it was applied
df_dropped = df_dropped.drop(columns=['Bsmt Unf SF_yeojohnson'], errors='ignore')
# Rename the inverse transformed column to the original name for consistency
# Ensure the inverse column exists before renaming
if 'Bsmt Unf SF_inverse' in df_dropped.columns:
df_dropped = df_dropped.rename(columns={'Bsmt Unf SF_inverse': 'Bsmt Unf SF'})

# Keep the inverse transformed 'Lot Frontage_log' and drop the original and log1p
transformed versions
# Note: We need to handle the case where the original 'Lot Frontage_log' and 'Lot
Frontage_log_log1p' might still be in the DataFrame
if 'Lot Frontage_log' in df_dropped.columns:
df_dropped = df_dropped.drop(columns=['Lot Frontage_log'], errors='ignore')
if 'Lot Frontage_log_log1p' in df_dropped.columns:
df_dropped = df_dropped.drop(columns=['Lot Frontage_log_log1p'], errors='ignore')
if 'Lot Frontage_log_yeojohnson' in df_dropped.columns: # Also check for
Yeo-Johnson
df_dropped = df_dropped.drop(columns=['Lot Frontage_log_yeojohnson'],
errors='ignore')

# Rename the inverse transformed column to 'Lot Frontage_transformed' or similar to
indicate transformation
# Ensure the inverse column exists before renaming
if 'Lot Frontage_log_inverse' in df_dropped.columns:
df_dropped = df_dropped.rename(columns={'Lot Frontage_log_inverse': 'Lot
Frontage_transformed'})

# --- Finalize transformations for other highly skewed columns ---
# Get the list of transformed columns to keep (excluding Bsmt Unf SF and Lot
Frontage_log as they are handled above)
transformed_cols_to_keep = [f"{col}_{data['Transformation Method'].lower()}" for
col, data in skewness_comparison.items() if col not in ['Bsmt Unf SF', 'Lot
Frontage_log']]

# Get the list of original highly skewed columns (excluding Bsmt Unf SF and Lot
Frontage_log)
original_highly_skewed_cols = [col for col in skewness_comparison.keys() if col not
in ['Bsmt Unf SF', 'Lot Frontage_log']]

# Drop the original highly skewed columns (excluding the two handled above)
df_dropped = df_dropped.drop(columns=original_highly_skewed_cols, errors='ignore')
# Use errors='ignore' in case some were already dropped

# Display the first few rows and the columns to confirm the changes
display("DataFrame head after finalizing transformations:")
display(df_dropped.head())
display("Columns in the DataFrame after finalizing transformations:")
display(df_dropped.columns)

```

[Receipt] Continuous Variable Transformation Summary

[Target] Objective:

To **normalize the distribution** of continuous numerical variables by **reducing skewness** and bringing them closer to a **bell-shaped (normal) curve**.

This ensures that **linear regression** and other statistical models can **learn efficiently** and **without bias**.

[Check] Outcome:

- Continuous features are now **statistically balanced** and **approximately normally distributed**.
- All **redundant original** and **interim transformation columns** were **dropped** to maintain a **clean and concise dataset structure**.
- [OK] The dataset is now fully prepared for the **Feature Scaling Stage**.

[Down]

[Green] Beginning the Encoding Process

[Puzzle] Step 1: Identify Categorical Variable Types

[Target] Subtask:

Identify which **categorical columns** belong to each type —

- **Nominal**: No inherent order (e.g., Neighborhood, House Style)
- **Ordinal**: Features with a ranked order (e.g., Exter Qual, Kitchen Qual)
- **Binary**: Two-level categorical variables (e.g., Street, Utilities)

This classification is based on each variable's **characteristics** and reference to the **dataset's data dictionary** (if available).

[Idea] Reasoning:

- The **first step** before encoding is to correctly identify the **type** of each categorical variable — whether it is **nominal, ordinal, or binary**.
 - This ensures that the **right encoding technique** is applied to each feature type.
 - The process involves:
 - Reviewing the **unique values** of each categorical column
 - Referring to the **data dictionary** for descriptions and logical order
 - Proper classification leads to **accurate encoding**, maintaining **data meaning and interpretability**.
-

[OK] Expected Outcome:

- Each categorical column will be **tagged** as nominal, ordinal, or binary.
 - This mapping will guide the **subsequent encoding steps** (Binary Ordinal One-Hot).
-

[Down] **Next**: Display the list of categorical columns.**

```
[72] Python
```

```
# Select only categorical columns from the DataFrame
```

```

categorical_cols = df_dropped.select_dtypes(include='object').columns
# Display the list of categorical columns
display("Categorical columns in the DataFrame:")
display(categorical_cols)
# Display unique values for each categorical column to help identify their type
for col in categorical_cols:
    display(f"Unique values for '{col}':")
    display(df_dropped[col].unique())
display("-" * 30) # Separator for readability

```

[Puzzle] Identify Binary Variables

Subtask: Identify categorical columns that contain only **two unique values**, indicating they are **binary variables** suitable for binary encoding.

[Down]

[73] Python

```

# Identify binary columns (those with exactly 2 unique values)
binary_cols = [col for col in categorical_cols if df_dropped[col].nunique() == 2]
# Display the identified binary columns
display("Identified binary categorical columns:")
display(binary_cols)

```

[Blue] Binary Variables

- **Columns:** Street, Utilities, Condition 2, Roof Matl, Heating, Central Air
- [Puzzle] **Explanation:** Binary variables have only **two possible values** (e.g., *Yes/No* or *0/1*).

These will encode using **simple binary mapping** to convert them into **numeric form** for modeling.

[Puzzle] Identify Ordinal and Nominal Variables

Subtask: From the remaining categorical columns, identify which features are **ordinal** (have an inherent order) and which are **nominal** (no natural order) to apply the correct encoding strategy.

[Down]

[74] Python

```

# Get the list of all categorical columns
all_categorical_cols = df_dropped.select_dtypes(include='object').columns.tolist()
# Get the list of binary columns (identified in the previous step)
# Assuming binary_cols variable from cell 72509ffc is available
# If not, re-run the binary identification step:
# binary_cols = [col for col in all_categorical_cols if df_dropped[col].nunique() == 2]

# Get the list of categorical columns that are not binary
non_binary_categorical_cols = [col for col in all_categorical_cols if col not in binary_cols]
# Display the non-binary categorical columns
display("Categorical columns that are not binary (likely Nominal or Ordinal):")
display(non_binary_categorical_cols)
# Display unique values for these columns again to help with classification
for col in non_binary_categorical_cols:
    display(f"Unique values for '{col}':")
    display(df_dropped[col].unique())

```


[Puzzle] Manual Classification of Remaining Categorical Variables

[Yellow] Ordinal Variables

- **Columns:**

Lot Shape, Land Slope, Exter Qual, Exter Cond, Bsmt Qual, Bsmt Cond,
Bsmt Exposure, BsmtFin Type 1, BsmtFin Type 2, Heating QC, Kitchen Qual,
Fireplace Qu, Garage Finish, Garage Qual, Garage Cond, Functional, Paved Drive

- **Explanation:**

Ordinal features have a **natural ranking or quality order** (e.g., *Poor < Fair < Good < Excellent*).
Hence, we apply **custom numeric mapping** to preserve their **inherent order and meaning**.

[Green] Nominal Variables

- **Columns:**

MS Zoning, Land Contour, Lot Config, Neighborhood, Condition 1, Bldg Type,
House Style, Roof Style, Exterior 1st, Exterior 2nd, Foundation, Electrical,
Garage Type, Sale Type, Sale Condition

- **Explanation:**

Nominal features have **no inherent order or ranking** — they are **pure labels**.
To prevent false numerical relationships, we use **One-Hot Encoding** for these variables.

[Puzzle] Encoding Categorical Variables (Binary, Ordinal, and Nominal)

[Target] Objective:

We now proceed to **encode categorical variables** into numerical format to prepare the dataset for machine learning models.

[Gear] Approach:

- **Binary Encoding:** For two-level categorical features (e.g., Street, Utilities, Roof Matl, Heating, Central Air)
 - **Ordinal Encoding:** For features with a natural order or ranking (e.g., Bsmt Qual, Kitchen Qual, Exter Qual, Garage Finish)
 - **One-Hot Encoding:** For nominal features with no intrinsic order (e.g., Neighborhood, House Style, Sale Type)
-

[Idea] Purpose:

- Convert all categorical data into **machine-readable numeric form**.
 - Maintain **semantic meaning** (especially for ordinal features).
 - Avoid introducing **bias or false ordering** among nominal categories.
-

[Down] **Next:** Begin binary encoding of two-level categorical variables.

```

# --- Encode Binary Variables ---
# Based on domain knowledge, defining binary mappings
binary_mappings = {
    'Street': {'Pave': 1, 'Others': 0}, # Pave is more common/desirable
    'Utilities': {'AllPub': 1, 'Others': 0}, # AllPub is the standard
    'Condition 2': {'Norm': 1, 'Others': 0}, # Normal condition is standard
    'Roof Matl': {'CompShg': 1, 'Others': 0}, # Standard roofing material
    'Heating': {'GasA': 1, 'Others': 0}, # Standard heating type
    'Central Air': {'Y': 1, 'N': 0} # Central air is a feature
}

for col, mapping in binary_mappings.items():
    df_dropped[col] = df_dropped[col].map(mapping)
display("DataFrame head after binary encoding:")
display(df_dropped.head())
display("Data types of binary encoded columns:")
display(df_dropped[list(binary_mappings.keys())].dtypes)

# --- Encode Ordinal Variables ---
# Based on domain knowledge and data dictionary, defining ordinal mappings
# Using custom mapping to preserve order
ordinal_mappings = {
    'Lot Shape': {'Reg': 4, 'IR1': 3, 'IR2': 2, 'IR3': 1}, # Regular to Irregular
    'Land Slope': {'Gtl': 3, 'Mod': 2, 'Others': 1}, # Gentle to Severe
    'Exter Qual': {'Ex': 5, 'Gd': 4, 'TA': 3, 'Fa': 2, 'Po': 1}, # Excellent to Poor
    'Exter Cond': {'Ex': 5, 'Gd': 4, 'TA': 3, 'Fa': 2, 'Po': 1}, # Excellent to Poor
    'Bsmt Qual': {'Ex': 6, 'Gd': 5, 'TA': 4, 'Fa': 3, 'Po': 2, 'None': 1}, # Excellent
    to Poor + No Basement
    'Bsmt Cond': {'Ex': 6, 'Gd': 5, 'TA': 4, 'Fa': 3, 'Po': 2, 'None': 1}, # Excellent
    to Poor + No Basement
    'Bsmt Exposure': {'Gd': 5, 'Av': 4, 'Mn': 3, 'No': 2, 'None': 1}, # Good Exposure
    to No Exposure + No Basement
    'BsmtFin Type 1': {'GLQ': 7, 'ALQ': 6, 'BLQ': 5, 'Rec': 4, 'LwQ': 3, 'Unf': 2,
    'None': 1}, # Good Living to Unfinished + No Basement
    'BsmtFin Type 2': {'GLQ': 7, 'ALQ': 6, 'BLQ': 5, 'Rec': 4, 'LwQ': 3, 'Unf': 2,
    'None': 1}, # Good Living to Unfinished + No Basement (using same scale as Type 1)
    'Heating QC': {'Ex': 5, 'Gd': 4, 'TA': 3, 'Fa': 2, 'Po': 1}, # Excellent to Poor
    'Kitchen Qual': {'Ex': 5, 'Gd': 4, 'TA': 3, 'Fa': 2, 'Po': 1}, # Excellent to Poor
    'Fireplace Qu': {'Ex': 6, 'Gd': 5, 'TA': 4, 'Fa': 3, 'Po': 2, 'None': 1}, #
    Excellent to Poor + No Fireplace
    'Garage Finish': {'Fin': 4, 'RFn': 3, 'Unf': 2, 'None': 1}, # Finished to
    Unfinished + No Garage
    'Garage Qual': {'Ex': 6, 'Gd': 5, 'TA': 4, 'Fa': 3, 'Po': 2, 'None': 1}, #
    Excellent to Poor + No Garage
    'Garage Cond': {'Ex': 6, 'Gd': 5, 'TA': 4, 'Fa': 3, 'Po': 2, 'None': 1}, #
    Excellent to Poor + No Garage
    'Functional': {'Typ': 8, 'Min1': 7, 'Min2': 6, 'Mod': 5, 'Maj1': 4, 'Maj2': 3,
    'Sev': 2, 'Sal': 1}, # Typical to Salvage
    'Paved Drive': {'Y': 3, 'P': 2, 'N': 1} # Paved to No Pavement
}

for col, mapping in ordinal_mappings.items():
    df_dropped[col] = df_dropped[col].map(mapping)
display("DataFrame head after ordinal encoding:")
display(df_dropped.head())
display("Data types of ordinal encoded columns:")
display(df_dropped[list(ordinal_mappings.keys())].dtypes)

```

```
# --- Encode Nominal Variables ---
# Using One-Hot Encoding for remaining categorical columns
# Get the list of all categorical columns remaining (excluding binary and ordinal
already encoded)
all_encoded_cols = list(binary_mappings.keys()) + list(ordinal_mappings.keys())
nominal_cols = [col for col in df_dropped.select_dtypes(include='object').columns
if col not in all_encoded_cols]

display("Nominal columns to be One-Hot Encoded:")
display(nominal_cols)

# Apply one-hot encoding
df_dropped = pd.get_dummies(df_dropped, columns=nominal_cols, drop_first=True) #
drop_first=True to avoid multicollinearity

display("DataFrame head after one-hot encoding:")
display(df_dropped.head())
display(f"Shape of the DataFrame after encoding: {df_dropped.shape}")

# Verify that there are no more object type columns
display("Remaining object type columns:")
display(df_dropped.select_dtypes(include='object').columns)
```

[Puzzle] Encoding Results Summary

[Num] 1. Binary Encoding Results:

- **Binary encoding** successfully applied to all binary categorical columns.
 - **Boolean columns** (Street, Utilities, Condition 2, Roof Matl, Heating, Central Air) were converted to **integer type (0 and 1)**.
 - [OK] Data verification confirmed that all encoded binary columns are of **dtype: int64**.
 - The **DataFrame** after binary encoding contained **72 columns**.
-

[Num] 2. Ordinal Encoding Results:

- **Ordinal variables** (e.g., Bsmt Qual, Bsmt Cond, Bsmt Exposure, BsmtFin Type 1, Fireplace Qu, Garage Finish, Garage Qual, Garage Cond, Functional) were successfully **mapped to integer scales** representing their rank order.
 - Some **NaN values** appeared in ordinal columns post-mapping; these were **handled later** (filled with value 1 in subsequent preprocessing).
 - Data verification confirmed correct encoding —
 - Ordinal columns are now primarily **int64 / float64**.
 - The **DataFrame** retained **72 columns** after ordinal encoding.
-

[Num] 3. One-Hot Encoding Results:

- **Nominal categorical columns** encoded via **one-hot encoding** included:

```
['MS Zoning', 'Land Contour', 'Lot Config', 'Neighborhood', 'Condition 1', 'Bldg Type', 'House
Style', 'Roof Style', 'Exterior 1st', 'Exterior 2nd', 'Foundation', 'Electrical', 'Garage Type',
'Sale Type', 'Sale Condition']
```

- Encoding expanded the dataset substantially — the **shape after one-hot encoding** became:

[OK] (2928 rows, 154 columns)

- All **object-type columns** were successfully transformed.
- Remaining object columns: `Index([], dtype='object')`
- Indicates **full categorical-to-numeric conversion** completed.

[Receipt] 4. Verification Summary:

- [OK] All **boolean columns converted** to integers.
- [OK] **Ordinal mappings verified** with expected numeric levels.
- [OK] **Nominal variables expanded** correctly through one-hot encoding.
- [OK] **No object-type columns remain**, ensuring **model compatibility**.
- [OK] Column names follow consistent format:

OriginalColumn_Category / Feature_TransformationSuffix — aiding interpretability and clarity.

[Up] Final Encoded DataFrame Overview:

- Shape: **(2928, 154)**
 - Categorical Numeric conversion: **100% complete**
 - Dataset now ready for:
 - **Further preprocessing tasks.**
-

[Down] **Next:** Proceeding to handle encoding-related issues (Task section).

[Down]

[Puzzle] Task: Handling Issues in Categorical Variable Encoding

[Target] Objective:

Address the issues identified during the **categorical variable encoding process**, including:

- Converting **boolean columns** to integer type
 - Handling **NaN values** in **ordinal columns**
 - Verifying **binary mappings** for 'Roof Matl' and 'Heating'
 - Ensuring **column name consistency** in the `df_dropped` DataFrame
-

[Num] Subtask 1: Convert Boolean Columns to Integers

[Gear] Description:

- Identify boolean columns generated through **one-hot encoding**.
 - Convert their data type from **boolean integer (0s and 1s)**.
 - Verify the conversion by checking the **data types** of all converted columns.
-

[Idea] Reasoning:

- The subtask ensures that **boolean columns** are stored as **integer values**, which is the preferred format for modeling libraries like **scikit-learn**.
- Converting booleans to integers improves:
- **Data consistency** across all features,
- **Compatibility** with model input requirements, and
- **Ease of data interpretation** during analysis.

[OK] Expected Outcome:

- All **boolean columns** successfully converted to **integers (0 and 1)**.
- Verified consistency of data types across the encoded dataset.
- Dataset ready for **further preprocessing** and modeling tasks.

[76] Python

```
# Identify boolean columns
boolean_cols = df_dropped.select_dtypes(include='bool').columns

# Convert boolean columns to integer type
for col in boolean_cols:
    df_dropped[col] = df_dropped[col].astype(int)

# Display the data types of the converted columns to verify the change
display("Data types of converted boolean columns:")
display(df_dropped[boolean_cols].dtypes)
```

Reasoning:

The boolean columns have been successfully converted to integers. The next step is to address the NaNs that were introduced in ordinal columns during the mapping process and fill them with an appropriate value, likely representing the absence of that feature or a default category.

[77] Python

```
# Identify ordinal columns that might have introduced NaNs
ordinal_cols_with_nans = [col for col in ordinal_mappings.keys() if
    df_dropped[col].isnull().any()]

# Fill NaNs in ordinal columns with the value representing 'None' or the lowest
category
# Based on the ordinal mappings, 'None' or the lowest category is typically mapped
to 1.
# However, we should confirm this or use a value that clearly indicates missingness
if 1 is a valid category.
# Let's check the unique values and mappings for these columns to be sure.
# Alternatively, we can fill with a value outside the defined range if appropriate.
# For simplicity and based on the previous mappings where 'None' was often mapped
to 1,
# we will fill NaNs with 1. If a column didn't have a 'None' category but has NaNs,
# filling with 1 might still be appropriate if 1 represents the lowest rank.
# Let's confirm which ordinal columns have NaNs first.
display("Ordinal columns with missing values after mapping:")
display(df_dropped[ordinal_cols_with_nans].isnull().sum())

# Fill NaNs in the identified ordinal columns with 1 (assuming 1 represents the
lowest rank or 'None')
for col in ordinal_cols_with_nans:
    df_dropped[col] = df_dropped[col].fillna(1)

# Verify that there are no more missing values in these ordinal columns
display("Missing values in ordinal columns after filling:")
display(df_dropped[ordinal_cols_with_nans].isnull().sum())
```

Reasoning:

The NaNs in the ordinal columns have been successfully filled. The next step is to verify the binary mappings for 'Roof Matl' and 'Heating' to ensure they were correctly encoded as 0s and 1s, as this was a potential issue identified in the overall task.

```
[78] Python
```

```
# Verify the unique values and data types for 'Roof Matl' and 'Heating'
display("Unique values and data type for 'Roof Matl' after encoding:")
display(df_dropped['Roof Matl'].unique())
display(df_dropped['Roof Matl'].dtype)
display("-" * 30)

display("Unique values and data type for 'Heating' after encoding:")
display(df_dropped['Heating'].unique())
display(df_dropped['Heating'].dtype)
```

Review column names

Subtask:

Ensure the column names are clear and consistent after all transformations and encodings.

Reasoning:

Display the current column names to visually inspect them.

```
[79] Python
```

```
# Display the current column names
display("Current columns in the DataFrame:")
display(df_dropped.columns.tolist())
```

[Receipt] Data Analysis Summary

[Chart] Key Findings:

- [OK] **Boolean columns** generated through **one-hot encoding** were successfully **converted to integer type (0s and 1s)**.
- [Brick] **Ordinal columns** —

(Bsmt Qual, Bsmt Cond, Bsmt Exposure, BsmtFin Type 1, BsmtFin Type 2, Fireplace Qu, Garage Finish, Garage Qual, Garage Cond, Functional) —

were identified to have **missing values (NaNs)** after mapping, and these were **filled with the value 1** to represent the lowest valid category.

- [Num] **Binary mappings** for 'Roof Matl' and 'Heating' columns were **verified** to contain only **0 and 1 values** with a correct **integer data type**.
- The **column names** in the df_dropped DataFrame were **reviewed and confirmed** to be:
- **Clear and consistent**,
- Using proper **suffixes for transformations**, and
- Following the **OriginalColumnName_Category** format for one-hot encoded variables.

[Idea] Insights / Next Steps:

- The process **successfully resolved all data consistency issues**, ensuring:
- **Correct data types**

- Proper handling of missing values
- Reliable structure for modeling
- The **clarity and consistency** of column names will make **future interpretation and manipulation** of the DataFrame **simpler and more efficient**.

[OK] Result:

The dataset was verified to be **well-structured, clean, and modeling-ready**, ensuring a smooth transition to the **feature scaling and modeling phases**.

[80] Python

```
# ■ Improved: create missing-indicators for ordinal columns that got NaNs during mapping
# and then fill those NaNs with the chosen rule (most -> 1, Functional -> 0).
# This block is safe: checks column existence before operating.
ordinal_cols_to_check_nans = [c for c in ordinal_mappings.keys() if c in df_dropped.columns]
# find which actually have NaNs
ordinal_cols_with_nans = [col for col in ordinal_cols_to_check_nans if df_dropped[col].isna().any()]
print("Ordinal columns identified with missing values:", ordinal_cols_with_nans)
# create indicators and fill NaNs with chosen rule
for col in ordinal_cols_with_nans:
    # indicator column
    indicator_col = f"{col}_missing"
    df_dropped[indicator_col] = df_dropped[col].isna().astype(int)
    # fill rule: for 'Functional' use 0, otherwise use 1 (as discussed)
    if col == 'Functional':
        df_dropped[col] = df_dropped[col].fillna(0)
    else:
        df_dropped[col] = df_dropped[col].fillna(1)
# quick verification
print("\nSample rows (showing original + indicator columns):")
cols_to_show = []
for col in ordinal_cols_with_nans:
    cols_to_show.extend([col, f"{col}_missing"])
display(df_dropped[cols_to_show].head(8))
print("\nVerify no NaNs remain in these ordinals:")
for col in ordinal_cols_with_nans:
    print(f"- {col}: remaining NaNs ->", df_dropped[col].isna().sum())
```

Checked for Remaining Missing Values After Ordinal Encoding

[Target] Reason for Performing This Step:

- To ensure that **no hidden missing values** remained after the **ordinal encoding process**.
- To determine whether **missing indicator variables** needed to be created for any ordinal features.

[Check] Finding:

- **No missing ordinal values were found** `ordinal_cols_with_nans = []`
- Therefore, **no missing indicator variables** were created.

- [OK] The ordinal feature set is **complete, consistent, and ready for further processing**.

[81] Python

```
# Save the DataFrame to a CSV file
df_dropped.to_csv('fully_cleaned_encoded_dataset.csv', index=False)
display("DataFrame saved to 'fully_cleaned_encoded_dataset.csv'")
```

[82] Python

```
import pandas as pd
# Load the saved CSV file
df_dropped = pd.read_csv('fully_cleaned_encoded_dataset.csv')
```

[83] Python

```
# ■ Final Dataset Verification – Column Types & Sanity Check
import pandas as pd
import numpy as np

print("■ Dataset Shape:", df_dropped.shape)
print("■ Total Columns:", len(df_dropped.columns))
print("-" * 60)

# Count columns by dtype
dtype_summary = df_dropped.dtypes.value_counts()
print("■ Column Type Summary:")
display(dtype_summary)

# Verify if any object dtype (should be none)
object_cols = df_dropped.select_dtypes(include='object').columns.tolist()
if len(object_cols) == 0:
    print("■ No object dtype columns remain – dataset fully numeric.")
else:
    print("■■ Object dtype columns found:", object_cols)

# Check first few rows to visually confirm encoding
print("-" * 60)
print("■ Sample of Final Encoded DataFrame:")
display(df_dropped.head())

# Optional: confirm target column exists
if 'SalePrice' in df_dropped.columns:
    print("■ Target column 'SalePrice' present and ready for modeling.")
else:
    print("■■ Target column 'SalePrice' not found – verify before modeling.")
```

[Down]

[Green] Variance Inflation Factor (VIF) Check for Multicollinearity

[Target] Objective:

- To **calculate the Variance Inflation Factor (VIF)** for all features in the dataset.
- This helps in identifying **multicollinearity** — where features are **highly correlated or overlapping** with one another.

[Brain] Purpose:

- Detect and evaluate **redundant variables** that carry similar information.
 - If such features are detected, they will be **dropped** to:
 - Prevent confusion in model interpretation, and
 - Enhance overall **model stability and performance**.
-

[OK] Expected Outcome:

- A **refined feature set** with **minimal redundancy**.
- Improved **model accuracy, interpretability, and efficiency** after removing high-VIF columns.

[84] Python

```
import pandas as pd
# Load the saved CSV file
df_dropped = pd.read_csv('fully_cleaned_encoded_dataset.csv')
```

[85] Python

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant
import pandas as pd
import numpy as np

# Assuming df_dropped is already loaded and contains the data
# Separate features (X) and target (y)
# Ensure 'SalePrice' is in the DataFrame before dropping it
if 'SalePrice' in df_dropped.columns:
    X = df_dropped.drop('SalePrice', axis=1)
    y = df_dropped['SalePrice']
else:
    display("Error: 'SalePrice' column not found in the DataFrame. Cannot separate
    features and target.")
    X = df_dropped # If SalePrice is not found, proceed with all columns for VIF check,
    but inform the user.
    y = None # Set y to None

# Add a constant to the features dataframe (required for VIF calculation by
statsmodels)
# Use has_constant='add' to handle cases where constant might already exist
X = add_constant(X, has_constant='add')

# Calculate VIF for each feature
vif_data = pd.DataFrame()
vif_data["feature"] = X.columns

# Ensure numerical data for VIF calculation
numerical_X = X.select_dtypes(include=np.number)

# Handle potential errors during calculation (e.g., perfect multicollinearity)
try:
    vif_data["VIF"] = [variance_inflation_factor(numerical_X.values, i)
    for i in range(numerical_X.shape[1])]
except Exception as e:
    display(f"An error occurred during VIF calculation: {e}")
    display("Please check for potential perfect multicollinearity (e.g., redundant
    dummy variables).")
    vif_data["VIF"] = float('nan') # Assign NaN if calculation fails

# Sort VIF values in descending order
vif_data = vif_data.sort_values(by="VIF", ascending=False)
```

```
# Display the VIF data
display("VIF factors for features:")
display(vif_data)
```

[86] Python

```
# Drop the original 'Lot Frontage' column
if 'Lot Frontage' in df_dropped.columns:
    df_dropped = df_dropped.drop(columns=['Lot Frontage'])
    display("Dropped the original 'Lot Frontage' column.")
else:
    display("'Lot Frontage' column not found in the DataFrame. Skipping drop.")

# Now, re-calculate VIF with the updated DataFrame
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

# Separate features (X) and target (y) from the updated DataFrame
# Ensure 'SalePrice' is still in the DataFrame before dropping it
if 'SalePrice' in df_dropped.columns:
    X = df_dropped.drop('SalePrice', axis=1)
    y = df_dropped['SalePrice']
else:
    display("Error: 'SalePrice' column not found in the DataFrame after dropping features.")
X = df_dropped # Proceed with all columns for VIF check if target is missing
y = None # Set y to None

# Add a constant to the features dataframe (required for VIF calculation by statsmodels)
# Use has_constant='add' to handle cases where constant might already exist
X = add_constant(X, has_constant='add')

# Calculate VIF for each feature
vif_data = pd.DataFrame() # Overwrite vif_data with the new calculation
vif_data["feature"] = X.columns

# Ensure numerical data for VIF calculation
numerical_X = X.select_dtypes(include=np.number)

# Handle potential errors during calculation
try:
    vif_data["VIF"] = [variance_inflation_factor(numerical_X.values, i)
                       for i in range(numerical_X.shape[1])]
except Exception as e:
    display(f"An error occurred during VIF calculation after dropping a feature: {e}")
    vif_data["VIF"] = float('nan') # Assign NaN if calculation fails

# Sort VIF values in descending order
vif_data = vif_data.sort_values(by="VIF", ascending=False)

# Display the VIF data after dropping a column
display("VIF factors for features after dropping 'Lot Frontage':")
display(vif_data)
```

[87] Python

```
# List of columns to drop
column_to_drop = 'Exterior 2nd_VinylSd'
display(f"Column to be dropped: '{column_to_drop}'")

# Drop this column from the DataFrame
# Ensure the column exists before dropping
if column_to_drop in df_dropped.columns:
```

```

df_dropped = df_dropped.drop(columns=[column_to_drop])
display(f"Dropped the column: '{column_to_drop}'.")
else:
display(f"Column '{column_to_drop}' not found in the DataFrame. Skipping drop.")

# Now, re-calculate VIF with the updated DataFrame
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

# Separate features (X) and target (y) from the updated DataFrame
# Ensure 'SalePrice' is still in the DataFrame before dropping it
if 'SalePrice' in df_dropped.columns:
X = df_dropped.drop('SalePrice', axis=1)
y = df_dropped['SalePrice']
else:
display("Error: 'SalePrice' column not found in the DataFrame after dropping
features.")
X = df_dropped # Proceed with all columns for VIF check if target is missing
y = None # Set y to None

# Add a constant to the features dataframe (required for VIF calculation by
statsmodels)
# Use has_constant='add' to handle cases where constant might already exist
X = add_constant(X, has_constant='add')

# Calculate VIF for each feature
vif_data = pd.DataFrame() # Overwrite vif_data with the new calculation
vif_data["feature"] = X.columns

# Ensure numerical data for VIF calculation
numerical_X = X.select_dtypes(include=np.number)

# Handle potential errors during calculation
try:
vif_data["VIF"] = [variance_inflation_factor(numerical_X.values, i)
for i in range(numerical_X.shape[1])]
except Exception as e:
display(f"An error occurred during VIF calculation after dropping a feature: {e}")
vif_data["VIF"] = float('nan') # Assign NaN if calculation fails

# Sort VIF values in descending order
vif_data = vif_data.sort_values(by="VIF", ascending=False)

# Display the complete VIF data table
display(f"Complete VIF factors for all features after dropping
'{column_to_drop}':")

# Use pandas option to display all rows
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
display(vif_data)

```

[88] Python

```

# List of columns to drop
column_to_drop = 'Roof Style_Hip'

display(f"Column to be dropped: '{column_to_drop}'")

# Drop this column from the DataFrame
# Ensure the column exists before dropping
if column_to_drop in df_dropped.columns:
df_dropped = df_dropped.drop(columns=[column_to_drop])
display(f"Dropped the column: '{column_to_drop}'.")
else:
display(f"Column '{column_to_drop}' not found in the DataFrame. Skipping drop.")

```

```

# Now, re-calculate VIF with the updated DataFrame
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

# Separate features (X) and target (y) from the updated DataFrame
# Ensure 'SalePrice' is still in the DataFrame before dropping it
if 'SalePrice' in df_dropped.columns:
    X = df_dropped.drop('SalePrice', axis=1)
    y = df_dropped['SalePrice']
else:
    display("Error: 'SalePrice' column not found in the DataFrame after dropping
    features.")
X = df_dropped # Proceed with all columns for VIF check if target is missing
y = None # Set y to None

# Add a constant to the features dataframe (required for VIF calculation by
statsmodels)
# Use has_constant='add' to handle cases where constant might already exist
X = add_constant(X, has_constant='add')

# Calculate VIF for each feature
vif_data = pd.DataFrame() # Overwrite vif_data with the new calculation
vif_data["feature"] = X.columns

# Ensure numerical data for VIF calculation
numerical_X = X.select_dtypes(include=np.number)

# Handle potential errors during calculation
try:
    vif_data["VIF"] = [variance_inflation_factor(numerical_X.values, i)
    for i in range(numerical_X.shape[1])]
except Exception as e:
    display(f"An error occurred during VIF calculation after dropping a feature: {e}")
    vif_data["VIF"] = float('nan') # Assign NaN if calculation fails

# Sort VIF values in descending order
vif_data = vif_data.sort_values(by="VIF", ascending=False)

# Display the complete VIF data table
display(f"Complete VIF factors for all features after dropping
'{column_to_drop}':")

# Use pandas option to display all rows
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    display(vif_data)

```

[89] Python

```

# List of columns to drop
column_to_drop = 'Sale Condition Partial'
display(f"Column to be dropped: '{column_to_drop}'")

# Drop this column from the DataFrame
# Ensure the column exists before dropping
if column_to_drop in df_dropped.columns:
    df_dropped = df_dropped.drop(columns=[column_to_drop])
    display(f"Dropped the column: '{column_to_drop}'.")
else:
    display(f"Column '{column_to_drop}' not found in the DataFrame. Skipping drop.")

# Now, re-calculate VIF with the updated DataFrame
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

# Separate features (X) and target (y) from the updated DataFrame

```

```

# Ensure 'SalePrice' is still in the DataFrame before dropping it
if 'SalePrice' in df_dropped.columns:
    X = df_dropped.drop('SalePrice', axis=1)
    y = df_dropped['SalePrice']
else:
    display("Error: 'SalePrice' column not found in the DataFrame after dropping
    features.")
X = df_dropped # Proceed with all columns for VIF check if target is missing
y = None # Set y to None

# Add a constant to the features dataframe (required for VIF calculation by
statsmodels)
# Use has_constant='add' to handle cases where constant might already exist
X = add_constant(X, has_constant='add')

# Calculate VIF for each feature
vif_data = pd.DataFrame() # Overwrite vif_data with the new calculation
vif_data["feature"] = X.columns

# Ensure numerical data for VIF calculation
numerical_X = X.select_dtypes(include=np.number)

# Handle potential errors during calculation
try:
    vif_data["VIF"] = [variance_inflation_factor(numerical_X.values, i)
    for i in range(numerical_X.shape[1])]
except Exception as e:
    display(f"An error occurred during VIF calculation after dropping a feature: {e}")
    vif_data["VIF"] = float('nan') # Assign NaN if calculation fails

# Sort VIF values in descending order
vif_data = vif_data.sort_values(by="VIF", ascending=False)

# Display the complete VIF data table
display(f"Complete VIF factors for all features after dropping
'{column_to_drop}':")

# Use pandas option to display all rows
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    display(vif_data)

```

[90] Python

```

# List of columns to drop
column_to_drop = 'Exterior 2nd_CemntBd'
display(f"Column to be dropped: '{column_to_drop}'")

# Drop this column from the DataFrame
# Ensure the column exists before dropping
if column_to_drop in df_dropped.columns:
    df_dropped = df_dropped.drop(columns=[column_to_drop])
    display(f"Dropped the column: '{column_to_drop}'.")
else:
    display(f"Column '{column_to_drop}' not found in the DataFrame. Skipping drop.")

# Now, re-calculate VIF with the updated DataFrame
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

# Separate features (X) and target (y) from the updated DataFrame
# Ensure 'SalePrice' is still in the DataFrame before dropping it
if 'SalePrice' in df_dropped.columns:
    X = df_dropped.drop('SalePrice', axis=1)
    y = df_dropped['SalePrice']
else:

```

```

display("Error: 'SalePrice' column not found in the DataFrame after dropping
features.")
X = df_dropped # Proceed with all columns for VIF check if target is missing
y = None # Set y to None

# Add a constant to the features dataframe (required for VIF calculation by
statsmodels)
# Use has_constant='add' to handle cases where constant might already exist
X = add_constant(X, has_constant='add')

# Calculate VIF for each feature
vif_data = pd.DataFrame() # Overwrite vif_data with the new calculation
vif_data["feature"] = X.columns

# Ensure numerical data for VIF calculation
numerical_X = X.select_dtypes(include=np.number)

# Handle potential errors during calculation
try:
vif_data["VIF"] = [variance_inflation_factor(numerical_X.values, i)
for i in range(numerical_X.shape[1])]
except Exception as e:
display(f"An error occurred during VIF calculation after dropping a feature: {e}")
vif_data["VIF"] = float('nan') # Assign NaN if calculation fails

# Sort VIF values in descending order
vif_data = vif_data.sort_values(by="VIF", ascending=False)

# Display the complete VIF data table
display(f"Complete VIF factors for all features after dropping
'{column_to_drop}':")

# Use pandas option to display all rows
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
display(vif_data)

```

[91] Python

```

# List of columns to drop
columns_to_drop = ['Exterior 2nd_MetalSd',
'1st Flr SF_log1p',
'2nd Flr SF_yeojohnson',
'Total Bsmt SF']

display(f"Columns to be dropped: {columns_to_drop}")

# Drop these columns from the DataFrame
# Ensure the columns exist before dropping
existing_columns_to_drop = [col for col in columns_to_drop if col in
df_dropped.columns]

if existing_columns_to_drop:
df_dropped = df_dropped.drop(columns=existing_columns_to_drop)
display(f"Dropped the columns: {existing_columns_to_drop}.")
else:
display("None of the specified columns were found in the DataFrame to drop.")

# Now, re-calculate VIF with the updated DataFrame
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

# Separate features (X) and target (y) from the updated DataFrame
# Ensure 'SalePrice' is still in the DataFrame before dropping it
if 'SalePrice' in df_dropped.columns:
X = df_dropped.drop('SalePrice', axis=1)
y = df_dropped['SalePrice']
else:

```

```

display("Error: 'SalePrice' column not found in the DataFrame after dropping
features.")
X = df_dropped # Proceed with all columns for VIF check if target is missing
y = None # Set y to None

# Add a constant to the features dataframe (required for VIF calculation by
statsmodels)
# Use has_constant='add' to handle cases where constant might already exist
X = add_constant(X, has_constant='add')

# Calculate VIF for each feature
vif_data = pd.DataFrame() # Overwrite vif_data with the new calculation
vif_data["feature"] = X.columns

# Ensure numerical data for VIF calculation
numerical_X = X.select_dtypes(include=np.number)

# Handle potential errors during calculation
try:
vif_data["VIF"] = [variance_inflation_factor(numerical_X.values, i)
for i in range(numerical_X.shape[1])]
except Exception as e:
display(f"An error occurred during VIF calculation after dropping features: {e}")
vif_data["VIF"] = float('nan') # Assign NaN if calculation fails

# Sort VIF values in descending order
vif_data = vif_data.sort_values(by="VIF", ascending=False)

# Display the complete VIF data table
display(f"Complete VIF factors for all features after dropping
{existing_columns_to_drop}:")

# Use pandas option to display all rows
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
display(vif_data)

```

[Chart] VIF (Multicollinearity) Analysis Summary

[Puzzle] Dropped Features (High Multicollinearity):

The following **seven highly collinear and redundant features** were removed to reduce multicollinearity and eliminate overlapping information:

```
['Lot Frontage', 'Roof Style_Hip', 'Sale Condition_Partial', 'Exterior 2nd_CemntBd', 'Exterior
2nd_MetalSd', '1st Flr SF_log1p', '2nd Flr SF_yeojohnson', 'Total Bsmt SF']
```

- These variables overlapped with other **structural or transformed predictors**, leading to inflated VIF values.
 - Removing them helps the model avoid **duplicate feature influence** and improves overall **stability and interpretability**.
-

[Scale] Retained Features with Slightly Elevated VIF:

- Certain variables (e.g., **Garage Area, Garage Cars, Neighborhood_***, **Bsmt Qual, Fireplace Qu**) had **VIF slightly above 5**.
- They were **retained** due to their **strong theoretical and domain relevance** to SalePrice.

Keeping them preserves essential **real-world explanatory power** without significant collinearity risk.

[Wood] Example – Exterior 2nd_Wd Sdng:

- Exterior 2nd_Wd Sdng = 1 House's **secondary exterior material is wood siding**.

Exterior_2nd_Wd_Sdng = 0 Material is **other type** (e.g., vinyl, cement board).

- This variable shows **VIF ≈ 9.7** , correlated with other one-hot exterior features (Exterior_2nd_VinylSd, Exterior_2nd_MetalSd, etc.) because they are **mutually exclusive**.
- Since drop_first=True was applied during encoding:
- The **redundancy effect is statistically handled**.
- Keeping this column is **safe and interpretable**, representing meaningful real-world material information.

[Brain] Interpretation:

- The feature Exterior_2nd_Wd_Sdng remains **statistically acceptable** and **valuable** despite slightly high VIF.
- Its inclusion enhances the model's **real-world interpretability** — reflecting **exterior material quality and style**.

[OK] Final Outcome:

- All **high multicollinearity features dropped** after thorough review.
- Dataset now has **no severe VIF issues** — ensuring:
- **Clean and stable predictor set**
- **Better model interpretability**
- **Reduced redundancy**
- [Check] **VIF Analysis Completed** — dataset ready for the next modeling stage.

[Green] Checking Correlation with Target Variable (`SalePrice`)

[Target] Subtask:

- Calculate the **correlation of each feature** with the target variable SalePrice.
- Identify the **most strongly correlated features** — both positively and negatively correlated ones.

[Idea] Reasoning:

- Correlation analysis helps to understand the **strength** and **direction** of the **linear relationship** between each feature and the **house sale price**.
- This step provides **insights into which variables have the highest influence** on price prediction.
- It also helps in **feature selection** — identifying which features may be **more predictive** and which contribute **less or add noise** to the model.

[OK] Outcome:

- A clear understanding of **which features drive sale prices the most**.
- Helps guide **model design** by focusing on **high-impact predictors** and managing weaker ones effectively.

```
[92] Python
```

```
import pandas as pd

# Calculate the correlation matrix
correlation_matrix = df_dropped.corr()

# Get the correlations with the target variable 'SalePrice'
# Drop the correlation of SalePrice with itself (which is always 1)
correlation_with_saleprice =
correlation_matrix['SalePrice'].sort_values(ascending=False)
correlation_with_saleprice = correlation_with_saleprice.drop('SalePrice')
```



```
# Display the correlations without truncation
display("Correlation of features with SalePrice (Sorted):")
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    display(correlation_with_saleprice)
```

[93] Python

```
# List of columns that I want to check the correlation for
columns_to_check = ['Foundation_Wood', 'Condition_1_RRAn', 'Neighborhood_Landmrk',
'Exterior_2nd_Others', '3Ssn_Porch', 'Utilities', 'Neighborhood_Greens',
'Neighborhood_SawyerW', 'Exterior_2nd_BrkFace', 'Lot_Config_FR3',
'House_Style_2.5Unf', 'Roof_Style_Shed', 'Neighborhood_GrnHill',
'House_Style_1.5Unf']

# Ensure correlation_with_saleprice Series is available from previous calculation
(cell 4a94bdc1)
if 'correlation_with_saleprice' in locals():
# Filter the correlation_with_saleprice Series for the specified columns
correlation_subset = correlation_with_saleprice[columns_to_check]
display("Correlation with SalePrice for the specified columns:")
display(correlation_subset)
else:
display("Correlation data with SalePrice not available. Please run the correlation
calculation code first.")
```

[94] Python

```
# List of columns to drop
columns_to_drop_low_corr = ['Foundation_Wood',
'Condition_1_RRAn',
'Neighborhood_Landmrk',
'Exterior_2nd_Others',
'3Ssn_Porch',
'Utilities',
'Neighborhood_Greens',
'Neighborhood_SawyerW',
'Exterior_2nd_BrkFace',
'Lot_Config_FR3',
'House_Style_2.5Unf',
'Roof_Style_Shed',
'Neighborhood_GrnHill',
'House_Style_1.5Unf']

display(f"Columns to be dropped due to low correlation:
{columns_to_drop_low_corr}")

# Drop these columns from the DataFrame
# Ensure the columns exist before dropping
existing_columns_to_drop = [col for col in columns_to_drop_low_corr if col in
df_dropped.columns]

if existing_columns_to_drop:
df_dropped = df_dropped.drop(columns=existing_columns_to_drop)
display(f"Dropped the columns: {existing_columns_to_drop}.")
else:
display("None of the specified low correlation columns were found in the DataFrame
to drop.")

# Display the shape of the DataFrame after dropping columns to confirm
display(f"Shape of the DataFrame after dropping low correlation columns:
{df_dropped.shape}")
```

[Cut] Dropped Columns Summary

[Receipt] Columns Removed:

```
['Foundation_Wood', 'Condition_1_RRAn', 'Neighborhood_Landmrk', 'Exterior_2nd_Others', '3Ssn_Porch',  
'Utilities', 'Neighborhood_Greens', 'Neighborhood_SawyerW', 'Exterior_2nd_BrkFace', 'Lot  
Config_FR3', 'House_Style_2.5Unf', 'Roof_Style_Shed', 'Neighborhood_GrnHill', 'House_Style_1.5Unf']
```

[Note] Reason for Removal:

- These columns showed **negligible correlation** with the target variable ($|r| < 0.05$).
 - Most represented **rare or low-variance categories**, contributing minimal predictive value.
 - Some variables were **redundant or extremely rare**, adding **noise** instead of useful signal.
 - Other low or negative correlation features were **retained** since they still convey **important structural details** (e.g., *house style*, *material type*) beneficial for model interpretation.
-

[OK] Resulting Benefits:

- The dataset is now **statistically cleaner** and **more stable**.
- **Noise and redundancy** have been reduced without losing **predictive strength**.
- Ensures a **leaner, well-optimized dataset** ready for model training and evaluation.

```
[95] Python
```

```
# Save the DataFrame to a CSV file  
df_dropped.to_csv('dataset_for_scaling.csv', index=False)  
display("DataFrame saved to 'dataset_for_scaling.csv'")
```

```
[96] Python
```

```
import pandas as pd  
# Load the saved CSV file  
df_dropped = pd.read_csv('dataset_for_scaling.csv')
```

[Green] Now We Are Going for Feature Scaling

```
[97] Python
```

```
import matplotlib.pyplot as plt  
import seaborn as sns  
import numpy as np  
  
# Get a list of all numerical columns in the current DataFrame  
all_numerical_cols_current =  
df_dropped.select_dtypes(include=np.number).columns.tolist()  
  
# Based on our previous analysis and transformations, let's identify the current  
# continuous columns  
# We need to consider the original continuous columns and their transformed  
# versions,  
# while excluding original discrete and encoded categorical columns.  
# Start with the list of original continuous columns  
current_continuous_cols = ['Lot Area', 'Mas Vnr Area', 'BsmtFin SF 1',  
                           'Bsmt Unf SF', 'Garage Area', 'Wood Deck SF', 'Open Porch SF']  
  
# Add the transformed continuous columns that we kept
```

```

# We need to check which transformed columns are currently in df_dropped
transformed_continuous_cols_kept = [
    col for col in all_numerical_cols_current
    if '_log1p' in col or '_yeojohnson' in col or '_transformed' in col
]

# Combine the lists, ensuring uniqueness and that they are actually in the current
DataFrame
purely_continuous_cols_in_df = list(set(current_continuous_cols +
transformed_continuous_cols_kept))
purely_continuous_cols_in_df = [col for col in purely_continuous_cols_in_df if col
in all_numerical_cols_current]

display("Identified purely continuous columns in the current DataFrame:")
display(purely_continuous_cols_in_df)

# Iterate through each identified purely continuous column and plot histogram and
boxplot side-by-side
print("Plotting histograms and boxplots for purely continuous columns...")
for col in purely_continuous_cols_in_df:
    plt.figure(figsize=(12, 5)) # Adjust figure size for side-by-side plots

    # Histogram
    plt.subplot(1, 2, 1) # 1 row, 2 columns, 1st plot
    sns.histplot(df_dropped[col], kde=True)
    plt.title(f'Histogram of {col}')
    plt.xlabel(col)
    plt.ylabel('Frequency')

    # Boxplot
    plt.subplot(1, 2, 2) # 1 row, 2 columns, 2nd plot
    sns.boxplot(y=df_dropped[col])
    plt.title(f'Boxplot of {col}')
    plt.ylabel(col)

    plt.tight_layout() # Adjust layout to prevent overlapping
    plt.show()

print("Finished plotting purely continuous column distributions.")

```

[Scale] Feature-wise Scaling Recommendations

Below is a summary of scaling choices based on feature distributions, skewness, and outlier presence:

-
- **[House] Bsmt Unf SF:**
 - Still **right-skewed** and contains **outliers** (as seen in the box plot).
 - **[OK] Recommended: RobustScaler**, since it's less sensitive to outliers and uses **median + IQR** for scaling.
-
- **[Tree] Wood Deck SF_yeojohnson:**
 - Distribution looks **close to normal** with **fewer outliers**.
 - **[OK] Recommended: StandardScaler** for effective normalization.
-
- **[Build] BsmtFin SF 1_yeojohnson:**
 - **Approximately normal distribution** with **minimal outliers**.
 - **[OK] Recommended: StandardScaler** (better normalization).
-
- **[Car] Garage Area:**
 - Slightly **left-skewed** with a few **outliers**.

- [OK] **Recommended: RobustScaler**, preferred due to mild outlier presence (StandardScaler also acceptable).

- **[Home] Lot Area:**

- Remains **right-skewed** and contains **several outliers**.
- [OK] **Recommended: RobustScaler** — best suited for heavy-tailed distributions.

- **[Brick] Mas Vnr Area_yeojohnson:**

- **Nearly normal** distribution with a few **potential outliers**.
- [OK] **Recommended: RobustScaler**, for safer scaling in presence of mild outliers.

- **[Ruler] Gr Liv Area_log1p:**

- After transformation, the feature is **symmetric and nearly normal**.
- [OK] **Recommended: StandardScaler** — ideal for normalized data.

- **[Window] Open Porch SF_yeojohnson:**

- **Fairly symmetric and near-normal**, but with **a few outliers** remaining.
- [OK] **Recommended: Either StandardScaler or RobustScaler** (both suitable).

- **[Plant] Lot Frontage_transformed:**

- Slightly **skewed (possibly left-skewed)** and may contain **outliers**.
 - [OK] **Recommended: RobustScaler** for better stability.
-

[Receipt] Summary

- **StandardScaler** Used for features with **normal or near-normal distributions**.
- **RobustScaler** Used for features with **outliers or skewed distributions**.
- Together, these scaling choices ensure **balanced normalization** and **model stability** across varying data ranges.

```
[98] Python
```

```
from sklearn.preprocessing import StandardScaler, RobustScaler

# Lists of purely continuous columns based on previous analysis and recommendations
standard_scale_cols = [
    'Wood Deck SF_yeojohnson',
    'BsmtFin SF 1_yeojohnson',
    'Gr Liv Area_log1p',
    'Mas Vnr Area_yeojohnson', # Recommended StandardScaler or RobustScaler, let's use
    StandardScaler for this group
    'Open Porch SF_yeojohnson' # Recommended StandardScaler or RobustScaler, let's use
    StandardScaler for this group
]

robust_scale_cols = [
    'Bsmt Unf SF',
    'Garage Area',
    'Lot Area',
    'Lot Frontage_transformed'
]

# Ensure columns exist in the DataFrame before scaling
standard_scale_cols_existing = [col for col in standard_scale_cols if col in
df_dropped.columns]
```

```

robust_scale_cols_existing = [col for col in robust_scale_cols if col in
df_dropped.columns]

display(f"Applying StandardScaler to columns: {standard_scale_cols_existing}")
display(f"Applying RobustScaler to columns: {robust_scale_cols_existing}")

# Create a copy to avoid modifying the original df_dropped directly if needed later
df_scaled = df_dropped.copy()

# Apply StandardScaler to the standard_scale_cols
if standard_scale_cols_existing:
    scaler_standard = StandardScaler()
    df_scaled[standard_scale_cols_existing] =
    scaler_standard.fit_transform(df_scaled[standard_scale_cols_existing])
    display("StandardScaler applied.")

# Apply RobustScaler to the robust_scale_cols
if robust_scale_cols_existing:
    scaler_robust = RobustScaler()
    df_scaled[robust_scale_cols_existing] =
    scaler_robust.fit_transform(df_scaled[robust_scale_cols_existing])
    display("RobustScaler applied.")

# Update df_dropped to the scaled version
df_dropped = df_scaled

display("DataFrame head after tailored feature scaling:")
display(df_dropped.head())

display("Descriptive statistics of scaled continuous columns:")
display(df_dropped[standard_scale_cols_existing +
robust_scale_cols_existing].describe())

```

[Gear] Feature Scaling Summary

- [OK] **All continuous features successfully scaled** to ensure consistent range and improved model performance.
- **Normally distributed features** were scaled using **StandardScaler**, resulting in:
 - Mean ≈ 0
 - Standard Deviation ≈ 1

Confirms **proper normalization** and uniform feature scaling.

- **Skewed or outlier-prone features** were scaled using **RobustScaler**, which:
 - Centers data around the **median**
 - Reduces the influence of **extreme values**

Ensures **stable and robust model performance** even with outliers.

- [Chart] **Final Outcome:**
 - Standard-scaled columns **Normalized around 0 (std = 1)**
 - Robust-scaled columns **Median-centered and less sensitive to outliers**
 - Overall Dataset now has **balanced feature magnitudes** ready for **model training**.

[99] Python

```

# Save the DataFrame to a CSV file
df_dropped.to_csv('dataset_for_modeling.csv', index=False)

display("DataFrame saved to 'dataset_for_modeling.csv'")

```

[Target] Final Visual Validation: Checking Scatter Plots of Top 10 Correlated Features with Sale Price

Now that all preprocessing steps — including **missing value handling**, **outlier capping**, **transformations**, and **feature scaling** —

have been completed, we are visualizing the **top 10 most correlated variables** with SalePrice using **scatter plots**.

Earlier, during the initial correlation analysis, we relied only on numerical correlation values because the data was **unclean and unscaled**,

and scatter plots at that stage would have shown **distorted or misleading relationships**.

At this final stage, scatter plots provide a **true visual confirmation** of how each key variable now aligns linearly with the target —

ensuring both **data integrity** and **model interpretability**. [OK]

```
[100] Python
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Ensure correlation_with_saleprice Series is available from previous calculation
# (cell 4a94bdc1)
# If not, re-run the correlation calculation code:
# correlation_matrix = df_dropped.corr()
# correlation_with_saleprice =
# correlation_matrix['SalePrice'].sort_values(ascending=False)
# correlation_with_saleprice =
# correlation_matrix['SalePrice'].drop('SalePrice').sort_values(ascending=False)

if 'correlation_with_saleprice' in locals():
    # Get the top 10 most correlated features (excluding SalePrice itself)
    top_10_correlated_features = correlation_with_saleprice.head(10)

    display("Top 10 features most correlated with 'SalePrice' and their correlation
    values:")
    display(top_10_correlated_features)

    # Create scatter plots for each of the top 10 features against 'SalePrice'
    print("\nGenerating scatter plots for the top 10 most correlated features vs.
    SalePrice:")
    for feature in top_10_correlated_features.index:
        plt.figure(figsize=(8, 5))
        sns.scatterplot(data=df_dropped, x=feature, y='SalePrice')
        plt.title(f'Scatter Plot of {feature} vs. SalePrice')
        plt.xlabel(feature)
        plt.ylabel('SalePrice')
        plt.show()
    else:
        display("Correlation data with SalePrice not available. Please run the correlation
        calculation code (cell 4a94bdc1) first.")
```

[Target] Final Correlation Validation using Scatter Plots

After completing all **data preprocessing**, **transformation**, and **feature scaling** steps,

scatter plots were generated for the **Top 10 most correlated variables with SalePrice** to visually validate the relationships.

[OK] These scatter plots clearly confirm a **strong positive linear relationship** between critical property attributes (such as **Overall Quality, Exterior Quality, Gr Liv Area, Kitchen Quality, Garage Capacity, and Year Built**) and the target variable **SalePrice**.

[Up] This final visualization confirms that the processed dataset now reflects accurate, interpretable, and realistic relationships — making it fully ready for the **modeling and prediction phase** in Phase 2 deliverables.

[No] Time Series Plots Are Not Applicable

Since the dataset does not contain any **time-dependent or sequential features**, **time series analysis and plotting techniques** are **not applicable** in this project. [Stop]

[Search] Exploratory Data Analysis (EDA) Summary

[Up] Insights and Patterns

- **[Brain] Strong Positive Correlations Identified:**
 - Certain property features show a **strong positive linear relationship** with the **Sale Price** of houses.
 - **Overall Material and Finish Quality (Overall Qual)** demonstrates the **highest correlation (~0.80)** — confirming it as the **most influential determinant** of property value.
 - **Exterior Quality (Exter Qual)** and **Living Area Size (Gr Liv Area_log1p)** also have high impact — indicating that **larger and better-finished homes command higher prices**.
 - **[House] Key Interior & Functional Attributes:**
 - **Kitchen Quality (Kitchen Qual)** and **Number of Bathrooms (Full Bath)** highlight that **comfort and usability directly influence buyer perception** and final pricing.
 - **Garage-related factors** — such as **Garage Cars, Garage Area, and Garage Finish** — emphasize the **importance of parking capacity and additional usable space**.
 - **Basement Quality (Bsmt Qual)** and **Year Built** indicate that **strong structural integrity and modern construction** significantly enhance property value.
 - **[Idea] Collective Insight:**
 - These **ten variables** together represent the **core structural, functional, and qualitative aspects** of residential properties.
 - They will likely act as **primary predictors** in the modeling phase — enabling the **machine learning model** to **understand and explain variations in housing prices more effectively**.
-

[Puzzle] Understanding Negative Correlations

- **Why They Matter:**
 - While **positively correlated features** explain what **increases** house prices, **negatively correlated ones** highlight the **factors that reduce** them.

- These features help the model **identify less desirable property traits** or **lower-valued characteristics** in the market.
- **Example & Importance:**
 - A model considering only **Overall Qual** or **Gr Liv Area** might **overestimate** prices for **older homes** or those with **detached garages**.
 - Including **negatively correlated features** like **Neighborhood_OldTown** or **Garage Type_Detchd** ensures **more realistic pricing**.
 - These variables help the model **learn downside factors** — what **depresses price** — preventing bias toward high-value predictors and **improving accuracy for low-priced properties**.
- **[Scale] Balanced Market Perspective:**
 - Considering **both positive and negative correlations** offers a **complete and balanced understanding** of the **real estate market dynamics**.

[Chart] Key Observations — Final Multicollinearity (VIF) Analysis

- **Constant Term:**
 - The **constant term (VIF $\approx$ 2.6M)** shows an **inflated value** (expected) and is **ignored** as it doesn't impact inter-feature relationships.
- **[OK] Acceptable Multicollinearity:**
 - Most **numerical and categorical variables** have **VIF < 10**, which is **within the acceptable range**, indicating **low interdependency**.
- **[Warn] Mildly Elevated VIFs:**
 - A few variables such as **MS SubClass**, **Neighborhood_NAMes**, **Exterior1st_HdBoard**, and **MS Zoning_RL** show **slightly higher VIF (~18–25)** — possibly due to **spatial or design similarities** among property types.
 - However, they remain **informative predictors** and are **retained** for their **domain relevance** in housing price prediction.
- **Categorical Relationships:**
 - Variables like **Neighborhood**, **Exterior Type**, and **Zoning Classification** naturally share **moderate correlations** because they reflect **location-based or architectural patterns**.
 - Despite overlap, they **capture essential real-estate variability** and thus **add valuable predictive power** to the model.

Sequential Summary of Work Done on Dataset

This analysis was performed on the **Ames Housing dataset** to understand the characteristics of the data, identify potential issues, and prepare it for **predictive modeling**.

[Receipt] Data Overview and Initial Inspection

- The dataset contains a mix of **numerical (integer and float)** and **categorical (object)** features.
 - Initial inspection revealed the presence of **missing values** and **potential outliers** in several columns.
-

[Tool] Missing Value Handling

- Columns with **>60% missing values** (Pool QC, Misc Feature, Alley, Fence, Mas Vnr Type) were **dropped** due to sparsity.
 - **Numerical columns** with lower missing percentages were imputed using appropriate strategies:
 - *Median* for continuous values like Lot Frontage.
 - *Zero* where missingness indicated absence (e.g., area/count variables).
 - **Categorical columns** were imputed with 'None' where missingness indicated absence (no basement, garage, or fireplace).
 - The single missing value in 'Electrical' was imputed with the **mode**.
 - [OK] After handling, **no missing values remain** in the dataset.
-

[Search] Duplicate Value Check

- No duplicate **rows or columns** were found.
-

[Down] Outlier Analysis and Handling

- Outliers in continuous variables were detected using the **IQR (Interquartile Range)** method.
 - Instead of removal, they were handled using **capping (upper/lower bounds)** to preserve data distribution.
 - Sparse features like Enclosed Porch, Screen Porch, and Pool Area were **dropped** due to minimal predictive value.
 - **Discrete outliers reviewed:**
 - Removed an extreme Bedroom AbvGr value (8 bedrooms).
 - Replaced invalid zero in Kitchen AbvGr with **median (1)**.
 - Removed unrealistic record with low TotRms AbvGrd and Gr Liv Area.
 - Retained high-end discrete outliers as genuine **luxury property** traits.
-

Categorical Feature Analysis and Encoding

- Checked categorical columns for **low-frequency categories** and **spelling inconsistencies**.
- Rare categories (<1%) were grouped as '**Others**' to improve model robustness and reduce one-hot encoding dimensionality.
- Corrected spelling errors (CmentBd CemntBd, Basment Basement, removed trailing spaces in 'Sale Type').

[Puzzle] Encoding Strategy:

- **Binary variables** (Street, Utilities, Condition 2, Roof Matl, Heating, Central Air) encoded as **0/1**.
 - **Ordinal variables** (Lot Shape, Land Slope, Exter Qual, Bsmt Qual, etc.) encoded using **custom numeric mapping** to preserve order.
 - Missing values during mapping were filled with **absence indicators** (e.g., 1 or 0).
 - **Nominal variables one-hot encoded**; boolean outputs converted to **integers (0/1)**.
-

Variable Transformation

- Analyzed **skewness** of continuous variables.
 - Applied **log1p** and **Yeo-Johnson** transformations to reduce skewness and approach normal distribution.
 - Applied **inverse transformations** for features that remained left-skewed (Bsmt Unf SF, Lot Frontage_log).
 - Retained the **most effective transformation** per variable.
-

[Cycle] Multicollinearity Check (VIF)

- Calculated **Variance Inflation Factor (VIF)** for all features.
 - Dropped **highly collinear features** (Lot Frontage, Roof Style_Hip, Sale Condition_Partial, Exterior 2nd_CemntBd, Exterior 2nd_MetalSd, 1st Flr SF_log1p, 2nd Flr SF_yeojohnson, Total Bsmt SF).
 - Retained features with mild VIF due to **strong domain relevance**.
-

[Up] Correlation with Target Variable

- Correlation of each feature with **'SalePrice'** calculated and analyzed.
 - Features with **near-zero correlation** were **dropped** to avoid noise and weak predictors.
-

[Gear] Feature Scaling

- **StandardScaler** applied to variables with approximately **normal distributions**.
 - **RobustScaler** used for variables still skewed or containing outliers — more resilient to extreme values.
-

[OK] Conclusion

The comprehensive **EDA and preprocessing pipeline** produced a **clean, structured, and well-transformed dataset**.

- Missing values, outliers, rare categories, and multicollinearity were addressed.
- Features are now **ready for robust predictive modeling** of house sale prices.
- Final dataset saved as `dataset_for_modeling.csv` for the next modeling phase.

```
[101] Python
```

```
import pandas as pd
# Load the dataset
df = pd.read_csv('/content/dataset_for_modeling finally.csv')
# Display the first few rows to confirm it's loaded
display(df.head())
```

Now we are going to show the heatmap with the top 15 highly correlated features

- After checking the scatter plots of the top 10 features, the heatmap helps us see a clearer overall picture.
- It shows how strongly each top feature is correlated with **SalePrice** in one visual view.
- The heatmap also reveals relationships among the top features themselves, which scatter plots cannot show.
- This step gives a more complete understanding of which features matter the most for price prediction.

```
[102] Python
```

```
import pandas as pd
# Calculate the correlation matrix
correlation_matrix = df.corr()
# Get the correlations with the target variable 'SalePrice'
# Drop the correlation of SalePrice with itself (which is always 1)
correlation_with_saleprice =
correlation_matrix['SalePrice'].sort_values(ascending=False)
correlation_with_saleprice = correlation_with_saleprice.drop('SalePrice')
```

```
# Display the top 15 correlated features (absolute value) without truncation
display("Top 15 features most correlated with 'SalePrice' (absolute value):")
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    display(correlation_with_saleprice.abs().sort_values(ascending=False).head(15))
```

[103] Python

```
import matplotlib.pyplot as plt
import seaborn as sns

# Get the top 15 features from the previously calculated correlation_with_saleprice
# Make sure to include 'SalePrice' itself for the correlation matrix
top_15_features = correlation_with_saleprice.abs().sort_values(ascending=False).head(15).index.tolist()
top_15_features.append('SalePrice')

# Calculate the correlation matrix for these top features
correlation_matrix_top_features = df[top_15_features].corr()

# Plot the heatmap
plt.figure(figsize=(14, 12)) # Adjust figure size for better readability
sns.heatmap(correlation_matrix_top_features, annot=True, cmap='coolwarm',
            fmt=".2f", linewidths=.5)
plt.title('Heatmap of Top 15 Features Correlation with SalePrice', fontsize=16)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```

[Chart] Heatmap Summary: Key Correlations with SalePrice

- **Overall Qual** shows the strongest correlation (0.79), meaning house quality is the biggest driver of SalePrice.
- **Exter Qual** (0.74) and **Gr Liv Area_log1p** (0.71) also strongly influence price, reflecting exterior quality and living area size as major factors.
- Other important predictors with strong correlations include **Garage Cars**, **Garage Area_log1p**, **Total Bsmt SF_log1p**, and **1st Flr SF_log1p** (all above 0.60).
- The heatmap confirms clear positive linear relationships between these top features and SalePrice.

[Brain] Insights

- These features should be prioritized during modeling because they hold the strongest predictive power.
- Some top features are highly correlated with each other (e.g., Garage Cars Garage Area, basement first floor area), indicating possible multicollinearity.

Dropping 'Garage Area' (Multicollinearity Fix)

- Garage Area and Garage Cars convey similar information and are strongly correlated.
- Garage Cars (correlation with SalePrice: ~0.6479) has a slightly higher absolute correlation than Garage Area (correlation with SalePrice: ~0.6454).
- Garage Cars is often a more direct and easily interpretable measure of a garage's utility (e.g., how many vehicles it can accommodate).
- Dropping Garage Area reduces multicollinearity and keeps the dataset cleaner for modeling.

[104] Python

```
# List of columns to drop
column_to_drop = 'Garage Area'

display(f"Column to be dropped: '{column_to_drop}'")
```

```
# Drop this column from the DataFrame
# Ensure the column exists before dropping
if column_to_drop in df.columns:
    df = df.drop(columns=[column_to_drop])
    display(f"Dropped the column: '{column_to_drop}'.")
else:
    display(f"Column '{column_to_drop}' not found in the DataFrame. Skipping drop.")
# Display the shape of the DataFrame after dropping columns to confirm
display(f"Shape of the DataFrame after dropping '{column_to_drop}': {df.shape}")
```

[Chart] Calculate Correlations with SalePrice (Updated Dataset)

[Puzzle] What We Need To Do

- Recalculate the Pearson correlation values for all features in the updated DataFrame.
- Identify the new top 15 features that have the strongest absolute correlation with *SalePrice*.
- Prepare these features for generating a fresh heatmap.

[Brain] Why This Step Is Important

- We removed **'Garage Area'** due to multicollinearity, which changed the structure of the dataset.
- To get accurate insights, we must recompute all correlations so the ranking of top features is based on the **current** dataset.
- This helps us see if any feature moved into the top 15 after removing 'Garage Area'.

[Gear] Outcome Expected

- Updated correlation list reflecting the new dataset.
- Correct top 15 features ready for heatmap visualization.
- Reliable basis for interpreting feature importance without multicollinearity interference.

[105] Python

```
import pandas as pd
# Calculate the correlation matrix for the updated DataFrame
correlation_matrix = df.corr()
# Get the correlations with the target variable 'SalePrice'
# Drop the correlation of SalePrice with itself (which is always 1)
correlation_with_saleprice =
correlation_matrix['SalePrice'].sort_values(ascending=False)
correlation_with_saleprice = correlation_with_saleprice.drop('SalePrice')
# Display the correlations without truncation
display("Correlation of features with SalePrice (Sorted):")
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    display(correlation_with_saleprice)
```

[Target]Result:

The correlations have been recalculated. Now, identify the top 15 features with the highest absolute correlation to *'SalePrice'* from the new *correlation_with_saleprice* to prepare for the heatmap visualization.

[106] Python

```
# Identify the top 15 features with the highest absolute correlation to 'SalePrice'
# Ensure 'SalePrice' itself is included for the heatmap
```

```

top_15_features = correlation_with_saleprice.abs().sort_values(ascending=False).head(15).index.tolist()
if 'SalePrice' not in top_15_features:
    top_15_features.append('SalePrice')

display("Top 15 features with highest absolute correlation to SalePrice (including SalePrice):")
display(top_15_features)

```

[Down] Now generating a heatmap to visualize the correlation matrix of the top 15 features (including 'SalePrice') identified in the previous step. This will provide a clear visual representation of their interrelationships.

[107] Python

```

import matplotlib.pyplot as plt
import seaborn as sns

# Get the top 15 features from the previously calculated correlation_with_saleprice
# Make sure to include 'SalePrice' itself for the correlation matrix
top_15_features = correlation_with_saleprice.abs().sort_values(ascending=False).head(15).index.tolist()
top_15_features.append('SalePrice')

# Calculate the correlation matrix for these top features
correlation_matrix_top_features = df[top_15_features].corr()

# Plot the heatmap
plt.figure(figsize=(14, 12)) # Adjust figure size for better readability
sns.heatmap(correlation_matrix_top_features, annot=True, cmap='coolwarm',
            fmt=".2f", linewidths=.5)
plt.title('Heatmap of Top 15 Features Correlation with SalePrice', fontsize=16)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

```

[Target] Final Task: Heatmap Findings After Dropping 'Garage Area'

[Search] What the Updated Heatmap Shows

- **Overall Qual** remains the strongest predictor (correlation ≈ 0.799) house quality drives price the most.
- **Exter Qual** (≈ 0.698) and **Gr Liv Area_log1p** (≈ 0.688) continue to be highly influential exterior quality and living space clearly impact value.
- **Kitchen Qual** (≈ 0.673) and **Garage Cars** (≈ 0.648) also remain strong predictors garage capacity is now the primary garage-related feature.
- Other features like **Bsmt Qual**, **Year Built**, **Garage Finish**, and **Full Bath** (all > 0.5) also play an important role.

[Chart] Key Findings from the Updated Dataset

- After removing 'Garage Area', the dataset now has **130 features**.
- **Garage Cars** now stands out more clearly as the dominant garage-related predictor.
- The heatmap confirms strong, clean linear relationships with SalePrice across the top features.
- Internal correlations (e.g., Year Built Garage Finish, Quality metrics Living Space) are still visible and should be monitored.

[Idea] Insights / Next Steps

- **Overall Qual**, **Exter Qual**, and **Gr Liv Area_log1p** remain the most important features for model building.

- Dropping **Garage Area** has reduced multicollinearity and kept the more interpretable feature (**Garage Cars**).
- Some feature groups still show internal correlation — future models may benefit from feature selection or dimensionality reduction to handle remaining multicollinearity.

[OK] Summary

- Quality and living space remain the strongest drivers of SalePrice.
- Garage Cars becomes the key garage-related predictor after removing Garage Area.
- The updated heatmap confirms stable, strong correlations with no major structural changes.
- Multicollinearity is reduced, making the dataset cleaner for modeling.

[Down][Down] Calculate Negative Correlations with SalePrice

What We Are Doing

- Compute Pearson correlation for all features with SalePrice.
- From these, identify the **top 15 most negatively correlated** features.
- Display their names along with correlation values.

[Brain] Why This Step Matters

- Negative correlations show which features decrease as SalePrice increases.
- This helps us understand which attributes might lower property value or move in the opposite direction.
- These features can provide valuable insights for model interpretation and feature engineering.

[Gear] Important Step Before Calculation

- Reload the correct DataFrame using the right filename to avoid outdated or missing data issues.
- Ensures correlation values are calculated on the **latest, updated dataset**.

[Down] Expected Outcome

- A clean list of the 15 strongest negative correlations.
- Clear understanding of which features work opposite to SalePrice.

[108] Python

```
import pandas as pd

# Calculate the Pearson correlation matrix for the entire DataFrame
correlation_matrix = df.corr()

# Extract the 'SalePrice' column from the correlation matrix
correlation_with_saleprice = correlation_matrix['SalePrice']

# Filter for negative correlations
negative_correlations = correlation_with_saleprice[correlation_with_saleprice < 0]

# Sort the negative correlations in ascending order
sorted_negative_correlations = negative_correlations.sort_values(ascending=True)

# Display the top 15 most negative correlations
display("Top 15 features with the highest negative correlation to 'SalePrice':")
with pd.option_context('display.max_rows', None, 'display.max_columns', None):
    display(sorted_negative_correlations.head(15))
```

[Down] Summary: Negative Correlations with SalePrice

[Search] Key Findings

- **Garage Type_Detchd** (≈ -0.366) Detached garages are linked with lower sale prices.
- **Foundation_CBlock** (≈ -0.351) Older/less preferred foundation types reduce price.
- **MS Zoning_RM** (≈ -0.293) Medium-density zones tend to sell for lower prices.

- **Lot Shape** (≈ -0.291) Irregular lot shapes are associated with reduced value.
- **Roof Style_Gable** (≈ -0.256) Common Gable roofs show slightly lower pricing.
- Neighborhood features like **OldTown**, **NAmes**, **IDOTRR**, **Edwards**, **BrkSide** (≈ -0.21 to -0.13) Older/less desirable neighborhoods show lower sale prices.
- **Sale Type_WD** (≈ -0.204) Conventional sales may correspond to lower property prices.
- **Wood Siding** (Exterior1st_Wd Sdng, Exterior2nd_Wd Sdng: ≈ -0.17) Wood siding tends to lower sale value compared to modern materials.

[Idea] Implications for Prediction

- These features represent conditions that **decrease** property value—important for modeling lower-priced homes.
- Including these variables helps create a **balanced model** that understands both positive and negative price drivers.
- Negative correlations ensure the model captures the **full market spectrum**, not just factors that raise prices.

[Down] Generating the Heatmap for Negative Correlations[Down]

Generating a heatmap for visualizing the correlation matrix of the top 15 negatively correlated features with 'SalePrice', including annotations and a color bar for clarity.

[109] Python

```
import matplotlib.pyplot as plt
import seaborn as sns

# Get the top 15 negatively correlated features from the previously calculated
sorted_negative_correlations
# Make sure to include 'SalePrice' itself for the correlation matrix
top_15_neg_correlated_features =
sorted_negative_correlations.head(15).index.tolist()
top_15_neg_correlated_features.append('SalePrice')

# Calculate the correlation matrix for these top features
correlation_matrix_neg_features = df[top_15_neg_correlated_features].corr()

# Plot the heatmap
plt.figure(figsize=(14, 12)) # Adjust figure size for better readability
sns.heatmap(correlation_matrix_neg_features, annot=True, cmap='coolwarm',
fmt=".2f", linewidths=.5)
plt.title('Heatmap of Top 15 Negatively Correlated Features with SalePrice',
fontsize=16)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```

[Down]Summary: Negative Correlations with SalePrice

[Search]Key Findings

- **Garage_Type_Detachd** (Detached Garage) shows the strongest negative correlation at approximately **-0.366**. This suggests that houses with detached garages tend to have lower sale prices compared to those with other garage types or no garage.
- **Foundation_CBlock** (Cinder Block Foundation) has a significant negative correlation of **-0.351**. This indicates that homes with older or less preferred foundation types are associated with lower sale prices.

- MS_Zoning_RM (Residential Medium Density) is negatively correlated at **-0.293**. This implies that properties in medium-density residential zones might fetch lower prices compared to other zoning types, possibly due to factors like lot size or property type.
- Lot_Shape has a negative correlation of **-0.291**. Since Lot_Shape was ordinally encoded with Reg (Regular) having the highest value and IR3 (Irregular) having the lowest, a negative correlation means that more irregular lot shapes are associated with lower sale prices.
- Roof_Style_Gable shows a negative correlation of **-0.256**. This indicates that houses with Gable roof styles, which are common, might be associated with lower prices compared to other, possibly more premium, roof styles.
- Several **Neighborhood-related features** (Neighborhood_OldTown, Neighborhood_NAmes, Neighborhood_IDOTRR, Neighborhood_Edwards, Neighborhood_BrkSide) also show notable negative correlations (ranging from -0.212 to -0.135). This suggests that properties in these specific older or less desirable neighborhoods are associated with lower sale prices.
- Sale_Type_WD (Warranty Deed - Conventional) is negatively correlated at **-0.204**. This is an interesting finding, as it implies that conventional sales might be associated with lower prices compared to other sale conditions, such as new construction sales.
- Exterior_1st_Wd_Sdng and Exterior_2nd_Wd_Sdng are kept despite their high similarity and collinearity, highlighting their distinct contributions to the model's understanding of price.
- Exterior_1st_Wd_Sdng and Exterior_2nd_Wd_Sdng (Wood Siding) show negative correlations of **-0.174** and **-0.168** respectively. This suggests that houses with wood siding might sell for less compared to homes with other, possibly more modern or durable, exterior materials.

Implications for Prediction

- These negatively correlated features provide crucial insights into factors that **depress house prices**. A model that effectively incorporates these variables will be better equipped to distinguish between properties at different price points and provide more accurate predictions, especially for lower-valued homes.
- Understanding these negative influences helps to build a more **balanced predictive model**, preventing it from solely focusing on features that drive prices up. This improves the model's ability to capture the full spectrum of market dynamics.

We are now entering the modeling phase, where the cleaned and engineered data will be used to train predictive models.

##[Tool] Before training, we will split the dataset into training and testing sets to evaluate how well our model generalizes.

```
[110] Python
import pandas as pd
df=pd.read_csv('Final_dataset_for_modeling.csv')
```

Preparing Data for Simple Linear Regression

What We Are Doing

- Selecting only the 'Gr Liv Area_log1p' feature
- Creating X_simple_train and X_simple_test from existing X_train and X_test
- Keeping y_train and y_test unchanged
- Ensuring both new datasets remain proper DataFrames

[Brain] Why This Step Is Important

- Simple Linear Regression requires only one input feature
- This helps us test how well a single predictor explains SalePrice
- Also allows comparison with the full Multiple Linear Regression model

[Gear] Process Summary

- Extract the single column from X_train X_simple_train
- Extract the same column from X_test X_simple_test
- Print shapes to confirm correct selection

```
[111] Python
X_simple_train = X_train[['Gr Liv Area_log1p']]
X_simple_test = X_test[['Gr Liv Area_log1p']]
display(f"Shape of X_simple_train: {X_simple_train.shape}")
display(f"Shape of X_simple_test: {X_simple_test.shape}")
```

Now Instantiating and Train Simple Linear Regression Model

Subtask:

Instantiating a new LinearRegression model and train that by using X_simple_train and y_train.

Reasoning:

I need to import the LinearRegression class, create a new instance of it named simple_linear_model, and then train this model using the X_simple_train and y_train datasets as specified in the subtask instructions.

```
[112] Python
from sklearn.linear_model import LinearRegression
# Create a new instance of the LinearRegression model
simple_linear_model = LinearRegression()
print("Training the simple Linear Regression model...")
simple_linear_model.fit(X_simple_train, y_train)
print("Simple Linear Regression model trained successfully.")
```

Evaluating Simple Linear Regression Model

Subtask:

Making predictions on the test set (X_simple_test) using the trained simple_linear_model and calculate evaluation metrics such as R-squared, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).

Reasoning:

The markdown block outlines the task of evaluating the simple linear regression model. The next step is to implement this evaluation by making predictions and calculating the specified metrics.

```
[113] Python
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
import numpy as np

# Make predictions on the test set
y_simple_pred = simple_linear_model.predict(X_simple_test)

# Calculate evaluation metrics
mae_simple = mean_absolute_error(y_test, y_simple_pred)
mse_simple = mean_squared_error(y_test, y_simple_pred)
rmse_simple = np.sqrt(mse_simple)
r2_simple = r2_score(y_test, y_simple_pred)

# Print the evaluation metrics
print("\n--- Simple Linear Regression Model Evaluation Metrics ---")
print(f"R-squared (R2): {r2_simple:.4f}")
print(f"Mean Absolute Error (MAE): {mae_simple:.2f}")
print(f"Mean Squared Error (MSE): {mse_simple:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_simple:.2f}")
```

Summary: Simple Linear Regression Model Performance

[Chart]Evaluation Metrics Overview

- **R-squared (R2):** 0.4874
- **Mean Absolute Error (MAE):** 42572.07
- **Mean Squared Error (MSE):** 4021767977.34
- **Root Mean Squared Error (RMSE):** 63417.41

*Initial Insights

- The R-squared value of 0.4874 indicates that approximately **48.74% of the variance** in SalePrice can be explained by the single feature 'Gr Liv Area_log1p'. This is a moderate correlation, suggesting that while living area is an important factor, it alone is not sufficient to fully explain house prices.
- The MAE of 42572.07 means that, on average, the model's predictions are off by about \$42,572.07. Similarly, the RMSE of 63417.41 provides a measure of the typical magnitude of the errors, penalizing larger errors more heavily.
- Comparing these metrics to the previous multiple linear regression model (R2: 0.8710, MAE: 19082.23, RMSE: 31814.41), it's clear that incorporating more features significantly improves prediction accuracy. The simple linear regression model performs considerably worse, highlighting the complexity of predicting house prices with just one variable.
- This subtask successfully instantiated, trained, and evaluated a simple linear regression model using only 'Gr Liv Area_log1p', providing a baseline for comparison with more complex models.

[Art]Visualize Simple Linear Regression Predictions

Subtask:

Creating a scatter plot of Actual vs Predicted SalePrice for the simple model, including the regression line.

Reasoning:

Generate a scatter plot comparing actual `y_test` values with predicted `y_simple_pred` values from the Simple Linear Regression model, and include a perfect prediction line for visual comparison.

[114] Python

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Create a scatter plot of actual vs. predicted SalePrice for the simple model
plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_test, y=y_simple_pred, alpha=0.6)

# Add a line representing perfect predictions (y = x)
# Determine the range for the perfect prediction line
min_val_simple = min(y_test.min(), y_simple_pred.min())
max_val_simple = max(y_test.max(), y_simple_pred.max())

plt.plot([min_val_simple, max_val_simple], [min_val_simple, max_val_simple],
color='red', linestyle='--', label='Perfect Prediction')

# Add titles and labels
plt.title('Simple Linear Regression: Actual vs. Predicted SalePrice', fontsize=16)
plt.xlabel('Actual SalePrice', fontsize=12)
plt.ylabel('Predicted SalePrice', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

[Down] The simple linear regression model consistently underestimates SalePrice, as most points fall below the perfect prediction line.

###**[Down]** Predictions for mid-range houses are tightly grouped, but high-priced homes show large errors and wide scatter.

###**[Down]** This happens because a single feature (Gr Liv Area_log1p) cannot capture the full complexity of house pricing.

###**[Down]** The plot visually confirms the low R and high error values of the simple model.

[Gear] Instantiate Linear Regression Model

[Tool] Subtask

- Import LinearRegression from sklearn.linear_model.
- Create an instance of the model so it's ready for training.

[Brain] Reasoning

- The model object must be instantiated before fitting it on our training data.
- This step initializes all default parameters of Linear Regression.

[115] Python

```
from sklearn.linear_model import LinearRegression

# Create an instance of the LinearRegression model
linear_model = LinearRegression()

print("Linear Regression model instantiated successfully.")
```

Result:

The Linear Regression model has been instantiated. The next step is to train the model using the prepared `X_train` and `y_train` datasets.

[Chart] Train-Test Split Data for Linear Regression Model

We will now split the dataset into training and testing sets.

- **Features (X):** All columns except 'SalePrice'.
- **Target (y):** The 'SalePrice' column.

This split is essential to evaluate the model's performance on unseen data and prevent overfitting. We will use a `test_size` of 20% and a `random_state` for reproducibility.

[116] Python

```
from sklearn.model_selection import train_test_split
import pandas as pd

# Separate features (X) and target (y)
X = df.drop('SalePrice', axis=1)
y = df['SalePrice']

# Perform the train-test split
# 80% for training, 20% for testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

# Display the shapes of the resulting datasets
display(f"Shape of X_train: {X_train.shape}")
display(f"Shape of X_test: {X_test.shape}")
display(f"Shape of y_train: {y_train.shape}")
display(f"Shape of y_test: {y_test.shape}")
```

[Lift]■Now going for train the model[Down]

[117] Python

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Assuming X_train, y_train are already defined from a previous split
# If linear_model is not already instantiated, do it here
if 'linear_model' not in locals():
    linear_model = LinearRegression()

print("Training the Linear Regression model...")
linear_model.fit(X_train, y_train)
print("Linear Regression model trained successfully.")

print("\n--- Evaluating Model Performance on Training Data ---")
# Make predictions on the training set
y_train_pred = linear_model.predict(X_train)

# Calculate evaluation metrics
mae_train = mean_absolute_error(y_train, y_train_pred)
mse_train = mean_squared_error(y_train, y_train_pred)
rmse_train = np.sqrt(mse_train)
r2_train = r2_score(y_train, y_train_pred)

# Print the evaluation metrics
print(f"R-squared (R2) on Training Data: {r2_train:.4f}")
```

```
print(f"Mean Absolute Error (MAE) on Training Data: {mae_train:.2f}")
print(f"Mean Squared Error (MSE) on Training Data: {mse_train:.2f}")
print(f"Root Mean Squared Error (RMSE) on Training Data: {rmse_train:.2f}")
```

>> Interpretation of Training Results

- The model shows strong performance on the training dataset.
- **R Score of 0.8834** indicates the model explains about 88% of the variance.
- **MAE of 17,257** shows the average prediction error is reasonably low.
- **RMSE of 26,464** indicates a moderate level of deviation from actual prices.
- Overall, the model demonstrates a good fit on the training data.

The Linear Regression model has been trained successfully

[Chart] Now going for Evaluate Linear Regression Model[Down]

[Target] Subtask

- Use the trained model to predict values on **X_test**.
- Compute key evaluation metrics: **R**, **MAE**, **MSE**, and **RMSE**.

[Brain] Reasoning

- After training, evaluating on unseen test data reveals the model's true performance.
- R shows how much variance is explained,

MAE and RMSE show how far predictions deviate from actual values.

- Requires importing metric functions from `sklearn.metrics` and `numpy` for `sqrt()`.

[118] Python

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# 1. Use the trained linear_model to make predictions on the X_test dataset
y_pred = linear_model.predict(X_test)

# 3. Calculate the Mean Absolute Error (MAE)
mae = mean_absolute_error(y_test, y_pred)

# 4. Calculate the Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_pred)

# 5. Calculate the Root Mean Squared Error (RMSE)
rmse = np.sqrt(mse)

# 6. Calculate the R-squared (R2) score
r2 = r2_score(y_test, y_pred)

# 7. Print all calculated evaluation metrics
print("\n--- Model Evaluation Metrics ---")
print(f"R-squared (R2): {r2:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Mean Squared Error (MSE): {mse:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
```

[Up] Summary of Model Evaluation Results

- **R = 0.87** Model explains **87% of the variation** in SalePrice — strong performance.

- **MAE $\approx$ 19,082** On average, predictions are off by **~19k**, which is quite accurate for housing data.
- **RMSE $\approx$ 31,814** Errors are generally moderate and stable, indicating no extreme deviation.
- Overall, the model delivers **reliable and consistent predictions**, suitable for the real estate industry.

[Art] Visualize Linear Regression Predictions

Subtask

- Create a scatter plot comparing **Actual vs Predicted** SalePrice values.
- Add a **perfect prediction line** to clearly show how close the predictions are.

[Brain] Reasoning

- This visualization helps us quickly judge model performance.
- If points lie close to the perfect line, predictions are accurate.
- The scatter plot reveals patterns like underprediction or overprediction.

[119] Python

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

# Create a scatter plot of actual vs. predicted SalePrice
plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.6)

# Add a line representing perfect predictions (y = x)
# Determine the range for the perfect prediction line
min_val = min(y_test.min(), y_pred.min())
max_val = max(y_test.max(), y_pred.max())

plt.plot([min_val, max_val], [min_val, max_val], color='red', linestyle='--',
label='Perfect Prediction')

# Add titles and labels
plt.title('Actual vs. Predicted SalePrice (Linear Regression)', fontsize=16)
plt.xlabel('Actual SalePrice', fontsize=12)
plt.ylabel('Predicted SalePrice', fontsize=12)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Final Model Evaluation Summary

1. Strong Overall Performance (R = 0.87)

Our Simple Linear Regression model explains approximately 87% of the variation in house prices. This indicates that the model captures the major patterns in the dataset effectively.

2. Accurate Predictions for Most Homes

Most average-priced houses are predicted close to their true values.

The Actual vs Predicted plot shows the points clustered around the ideal prediction line, confirming good baseline performance.

3. Reasonable Error Levels

MAE $\approx$ 19,000

RMSE $\approx$ 31,800

These metrics show that the model maintains a stable and acceptable error range for this dataset.

4. Underestimation for High-Priced Houses

The model slightly underestimates luxury or high-priced homes, which appear more scattered.

This happens because expensive houses often have complex, nonlinear patterns that simple linear regression cannot fully capture.

5. No Major Bias or Structural Issue

There is no clear pattern in the residuals, meaning the model is not overfitting or underfitting.

Errors appear random, which is a positive sign.

6. Possible Future Improvement

For better prediction of premium houses, we can introduce additional feature engineering such as:

- combining ground area with luxury feature indicators,
- creating interaction features,
- or scaling certain property-size features.

These steps may help the model better capture the characteristics of high-value homes.

7. Final Verdict

The model is stable, reliable, and fully suitable for the scope of this Apex Project.

It demonstrates a clear understanding of:

linear regression,

dataset behavior,

model performance,

and limitations with meaningful improvement ideas.

[Target] Objective Comparison Analysis between Simple and Multiple Linear Regression

- Summarize Simple Linear Regression (SLR) performance using 'Gr Liv Area_log1p'.
- Compare SLR with Multiple Linear Regression (MLR).

[Chart] Simple Linear Regression (SLR) - Single Feature

- **Feature Used:** 'Gr Liv Area_log1p' (Log-transformed living area).
- **Performance:**
 - **R:** 0.4874 (explains ~49% of price variation).
 - **MAE:** ~\$42,572 (average prediction error).
 - **RMSE:** ~\$63,417 (errors spread widely).
- **Interpretation:**
 - Useful, but insufficient alone to capture housing price complexity.
 - Struggles with varied home sizes and conditions.

[Up] Multiple Linear Regression (MLR) - All Features

- **Performance:**
- **R:** 0.8710 (explains ~87% of price variation).
- **MAE:** ~\$19,082.
- **RMSE:** ~\$31,814.
- **Interpretation:**
- Captures relationships across many predictors.
- Errors are almost half compared to SLR.
- More stable and accurate predictions overall.

SLR vs. MLR - Key Differences

- **SLR (1 Feature):** Lower R (0.48), higher errors (MAE ≈ \$42.5k, RMSE ≈ \$63.4k).
- **MLR (Many Features):** Higher R (0.87), significantly lower errors (MAE ≈ \$19k, RMSE ≈ \$31.8k).

[Search] Key Insights

- House prices depend on *many* factors, not just living area.
- Single-feature models oversimplify the problem.
- MLR captures richer, more complex relationships.

[Flag] Final Conclusion

- Simple Linear Regression (SLR) is **insufficient** for accurate house price prediction.
- Multiple Linear Regression (MLR) **clearly outperforms** SLR.
- Accurate house price prediction **requires several relevant features**.

[Red]Now we are going to perform Residual Analysis

- Residuals help us understand how far the model's predictions are from the actual values.
- This step checks whether the model is leaving any hidden pattern unexplained.
- A good model should have residuals that look random, not forming any curve or shape.
- By examining residuals, we can confirm if the simple linear regression model is behaving correctly or showing bias.

[120] Python

```
# Residuals
residuals = y_test - y_pred

# 1) Residuals vs Predicted Values
plt.figure(figsize=(6,4))
plt.scatter(y_pred, residuals, alpha=0.6)
plt.axhline(0, color='red', linestyle='--')
plt.xlabel("Predicted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs Predicted Values")
plt.show()

# 2) Residual Distribution
plt.figure(figsize=(6,4))
sns.histplot(residuals, kde=True)
plt.xlabel("Residuals")
plt.title("Residual Distribution")
```



```
plt.show()
```

[Search] Residual Analysis

1. Residuals vs Predicted Values

The residuals are scattered around the zero line without forming any clear curve or pattern. This indicates that the model has captured the linear trend reasonably well. The majority of residuals lie close to zero, showing consistency across mid-range house prices.

2. Slight Increase in Spread for High-Priced Houses

At higher predicted values, the residuals become more spread out. This means that the model struggles slightly with expensive homes, which is normal because premium houses often have complex, nonlinear patterns that simple linear regression cannot fully capture.

3. Homoscedasticity Check

There is no funnel shape or systematic widening pattern. Most points show a fairly uniform spread, which suggests that the variance of errors is mostly stable (near-homoscedastic). This is acceptable for a linear regression model.

4. Residual Distribution

The histogram of residuals is roughly symmetric and centered around zero. Although not a perfect bell curve, the overall shape is close to a normal distribution. This indicates that the model's errors are not heavily skewed or biased.

5. Outliers

A few extreme residuals exist (very high or very low), mainly corresponding to luxury houses or unusual properties. These outliers do not significantly affect the overall model performance.

6. Final Interpretation

The residual plots confirm that the model performs well for most homes, with stable errors and no major violations of linear regression assumptions. Only high-end houses show higher variance, which is expected and clearly noted in the model conclusion.

Moving Ahead: Training More Models to Find the Best Performer

##To further improve our results, we will now train and test additional models.

###This will help us identify whether another algorithm can perform better on this dataset and achieve higher accuracy than Linear Regression.

>> Now we are going to try Ridge regression model to see if adding regularization could improve stability and reduce overfitting beyond the basic Linear Regression model.[Down]

[121] Python

```
from sklearn.linear_model import Ridge
# Create Ridge Regression model
ridge_model = Ridge()
print("Ridge Regression model instantiated successfully.")
```

[122] Python

```
from sklearn.linear_model import Ridge
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np
# Instantiate model
ridge_model = Ridge()
# Train the model
ridge_model.fit(X_train, y_train)
print("Ridge Regression model trained successfully.\n")
# Predictions on test data
y_pred_ridge = ridge_model.predict(X_test)
# Evaluation metrics
print("--- Ridge Regression Model Evaluation (Test Data) ---")
print("R-squared (R2):", r2_score(y_test, y_pred_ridge))
print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred_ridge))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred_ridge))
print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(y_test,
y_pred_ridge)))
```

These training results show that Ridge Regression fits the training data well, similar to the Linear model. The error values and R score indicate stable performance without major deviation from the baseline model.

[123] Python

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
# Assuming ridge_model is already trained and X_test, y_test are available
# Use the trained ridge_model to make predictions on the X_test dataset
y_pred_ridge = ridge_model.predict(X_test)
# Calculate the Mean Absolute Error (MAE)
mae_ridge = mean_absolute_error(y_test, y_pred_ridge)
# Calculate the Mean Squared Error (MSE)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
# Calculate the Root Mean Squared Error (RMSE)
rmse_ridge = np.sqrt(mse_ridge)
# Calculate the R-squared (R2) score
r2_ridge = r2_score(y_test, y_pred_ridge)
# Print all calculated evaluation metrics
print("\n--- Ridge Model Evaluation Metrics on Test Data ---")
print(f"R-squared (R2): {r2_ridge:.4f}")
print(f"Mean Absolute Error (MAE): {mae_ridge:.2f}")
print(f"Mean Squared Error (MSE): {mse_ridge:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse_ridge:.2f}")
```

These results show that Ridge Regression performs consistently on unseen test data.

The scores confirm that the model maintains similar accuracy to the Linear model without any significant improvement or drop.

>> Now we are going to try Lasso Regression model to check if shrinking unnecessary coefficients can simplify the model and improve performance beyond Ridge and Linear Regression. [Down]

[124] Python

```
from sklearn.linear_model import Lasso
# Create Lasso Regression model
lasso_model = Lasso()
print("Lasso Regression model instantiated successfully.")
```

[125] Python

```
from sklearn.linear_model import Lasso
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np
# Instantiate model
lasso_model = Lasso()
# Train the model
lasso_model.fit(X_train, y_train)
print("Lasso Regression model trained successfully.\n")
# Predictions on training data
y_pred_train_lasso = lasso_model.predict(X_train)
# Evaluation metrics
print("--- Lasso Model Evaluation Metrics on Training Data ---")
print("R-squared (R2):", r2_score(y_train, y_pred_train_lasso))
print("Mean Absolute Error (MAE):", mean_absolute_error(y_train, y_pred_train_lasso))
print("Mean Squared Error (MSE):", mean_squared_error(y_train, y_pred_train_lasso))
print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(y_train, y_pred_train_lasso)))
```

These training metrics indicate that the Lasso model fits the training data closely, very similar to the Ridge and Linear models.

The low error values and high R confirm that the model is performing reliably on the training dataset.

[126] Python

```
# Predict on test data
y_pred_test_lasso = lasso_model.predict(X_test)
# Evaluation metrics
print("--- Lasso Model Evaluation Metrics on Test Data ---")
print("R-squared (R2):", r2_score(y_test, y_pred_test_lasso))
print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred_test_lasso))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred_test_lasso))
print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(y_test, y_pred_test_lasso)))
```

These results show that the Lasso model performs consistently on unseen data with accuracy similar to the Linear and Ridge models.

The test metrics confirm that regularization did not significantly change performance, and the model generalizes in a stable manner.

>> Now we are going to try Random Forest Regression to explore a more powerful non-linear model and see if it can achieve higher accuracy than the previous linear models.
[Down]

[127] Python

```
from sklearn.ensemble import RandomForestRegressor
# Create Random Forest Regression model
rf_model = RandomForestRegressor()
print("Random Forest model instantiated successfully.")
```

[128] Python

```
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np
# Instantiate model
rf_model = RandomForestRegressor()
# Train the model
rf_model.fit(X_train, y_train)
print("Random Forest model trained successfully.\n")
# Predictions on training data
y_pred_train_rf = rf_model.predict(X_train)
# Evaluation metrics
print("--- Random Forest Model Evaluation Metrics on Training Data ---")
print("R-squared (R2):", r2_score(y_train, y_pred_train_rf))
print("Mean Absolute Error (MAE):", mean_absolute_error(y_train, y_pred_train_rf))
print("Mean Squared Error (MSE):", mean_squared_error(y_train, y_pred_train_rf))
print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(y_train,
y_pred_train_rf)))
```

These training metrics show that the Random Forest model has achieved extremely high accuracy on the training data.

The very low error values and near-perfect R indicate that the model has learned the training patterns in great depth.

[129] Python

```
# Predict on test data
y_pred_test_rf = rf_model.predict(X_test)
# Evaluation metrics
print("--- Random Forest Model Evaluation Metrics on Test Data ---")
print("R-squared (R2):", r2_score(y_test, y_pred_test_rf))
print("Mean Absolute Error (MAE):", mean_absolute_error(y_test, y_pred_test_rf))
print("Mean Squared Error (MSE):", mean_squared_error(y_test, y_pred_test_rf))
print("Root Mean Squared Error (RMSE):", np.sqrt(mean_squared_error(y_test,
y_pred_test_rf)))
```

These results show a significant improvement in accuracy on the test data compared to the previous models.

The higher R and noticeably lower error values indicate that Random Forest is the strongest performer on unseen house prices.

[Chart] Model Performance Comparison (Test Data)

Model	R-Squared (R)	MAE ()	MSE	RMSE ()	Notes
Simple Linear Regression	~0.487	~42572	very high	~63417	Weak model, single feature only
Multiple Linear Regression	~0.871	~19082	high	~31814	Strong baseline model
Ridge Regression	~0.870	~19165	high	~31862	Similar to Linear
Lasso Regression	~0.870	~19157	high	~31835	Similar to Linear
Random Forest Regression	~0.919	~15292	low	~25244	Best performance

>> Why We Used Ridge, Lasso & Random Forest After Linear Regression?

After training and evaluating the Multiple Linear Regression model, we decided to explore more models for three main reasons:

[OK] 1) To Reduce Possible Overfitting

Linear Regression can sometimes overfit the training data, especially when:

- the number of features is high,
- or some features are correlated.

So, we tested **Ridge** and **Lasso** to apply regularization and see whether performance improves on the unseen Test Data.

[OK] 2) To Improve Model Stability & Coefficient Control

- **Ridge Regression** reduces large coefficients and stabilizes the model.
- **Lasso Regression** also performs feature selection by shrinking unimportant coefficients.

The goal was to check:

- Does the model get more stable?
- Do errors reduce?
- Does Test accuracy increase?

[OK] 3) To Compare With a Non-Linear & More Powerful Model

Linear, Ridge, and Lasso are all **linear models**.

Therefore, we tested **Random Forest Regressor** to:

- capture complex non-linear patterns,
 - improve predictive power,
 - and reduce errors further.
-

[Search] What We Expected

- Ridge & Lasso may slightly improve Test performance.
 - Random Forest may perform significantly better due to non-linear capability.
-

What We Observed

Model	Test R	MAE	RMSE
Linear Regression	~0.871	high	high
Ridge	~0.870	similar	similar
Lasso	~0.870	similar	similar
Random Forest	~0.919	lowest	lowest

Findings:

- Ridge & Lasso performed very similar to Linear Regression.
 - Random Forest clearly delivered the best performance on Test Data.
 - Errors reduced significantly with Random Forest.
 - Generalization improved.
-

[Flag] Final Conclusion

We tried multiple models after Linear Regression to validate and improve our results.

Ridge & Lasso confirmed that our Linear model was already well-balanced,
while Random Forest achieved the best prediction accuracy on unseen data.

[Trophy] Random Forest emerges as the strongest model, delivering the highest accuracy and lowest errors on unseen data. [Win]

End of Report

This document was generated from the Jupyter Notebook submission

The_Extractors_Real_Estate_Project_Apex_Final_Submission.ipynb.

It covers the complete preprocessing, EDA, feature engineering, and modelling pipeline for the Ames Housing Dataset price prediction task.